

四川大學

SICHUAN UNIVERSITY

本科生毕业设计（学术论文）

Undergraduate Graduation Project
(Academic Thesis)



题 目 (Title) Threshold Transfer in Prompt-Layer Jailbreak
and Prompt Injection Detection for LLM Applications

学 院 (College) College of Software Engineering

专 业 (Major) Software Engineering

学生姓名 (Student Name) Rida BOUBAKR

学 号 (Student Id) 2022521460139 年 级 (Batch) 2022

指导教师 (Adviser) Chen Yanru

教务处制表

2026 年 4 月 20 日

Made by the Office of Academic Affairs

April 20, 2026

ABSTRACT

Software Engineering

Student: Rida BOUBAKR Adviser: Chen Yanru

[Abstract] Large language model applications often construct final model inputs by combining user prompts with retrieved or attached context, which creates a practical attack surface for jailbreaks and prompt injection. Existing guardrail evaluation often emphasizes threshold-free ranking metrics, but it addresses less directly whether a detector calibrated for low benign false-positive rate remains usable when prompt distributions shift.

This thesis studies that gap through a reproducible empirical case study of an offline prompt-layer detector that scores one composed prompt+context payload before the downstream LLM call. The implemented system combines a rules-based baseline with an optional LoRA-adapted DeBERTa-v3 detector, and it is evaluated under a locked threshold-transfer protocol: a decision threshold is calibrated on validation at approximately 1% benign false-positive rate and then transferred unchanged to selection-aware stress splits and fixed perturbation suites.

Results show that the selected learned detector preserves strong ranking on the main stress split, reaching AUROC = 0.9958, but the transferred operating point is unstable under shift. Benign false-positive rate rises from 0.0094 on validation to 0.0311 on the main stress split, 0.5816 on the JBB stress split, and 0.6403 on test_main_adv2. These findings show that strong ranking does not by itself guarantee stable low-FPR decision behavior under distribution shift in the implemented pipeline.

The main contribution of the thesis is an auditable case study and evaluation protocol that treats threshold choice as a monitored design object in prompt-layer guardrails, rather than as a secondary reporting detail.

[Key Words] large language models, prompt injection, jailbreak detection, guardrails, threshold calibration, robustness evaluation

摘要

软件工程

学生：Rida BOUBAKR 指导老师：Chen Yanru

大语言模型应用通常会将用户提示与检索得到或附加的上下文拼接后再送入模型，这使提示层同时暴露于越狱攻击与提示注入攻击。本文将研究范围限定为一个已实现的提示层检测管线在单一组合文本载荷上的可复现实证个案研究。本文关注的不是仅报告阈值无关的排序能力，而是考察一个在验证集上以约 1% 良性误报率校准得到的决策阈值，能否在不重新调参的情况下稳定迁移到选择感知的发展性压力评估划分与固定扰动套件上。

实验结果表明，所选学习型检测器在主压力划分（`test_main`）上具有很强的排序能力，AUROC 达到 0.9958；但该迁移后的决策阈值在分布偏移下并不稳定。良性误报率由验证集上的 0.0094 上升至主压力划分上的 0.0311、JBB 压力划分（`test_jbb`）上的 0.5816，以及 `test_main_adv2` 上的 0.6403。阈值邻域扫描、推理阶段规范化检查、良性占比更高条件下的再校准，以及冻结提交后的三种随机种子复现实验，都在一定程度上削弱了将该结果简单归结为单一实现伪影的最直接解释，但并未将其完全排除。

这些跟进分析并不能说明预处理不匹配、截断或其他管线因素不起作用；它们很可能仍然参与了所观察到的不稳定性。因而，本文所支持的是一个更为收敛、且依赖具体研究设置的经验性解释：在本文实现的数据混合方式、选择感知的发展性压力评估设置以及冻结配置之下——其中包括训练阶段与推理阶段规范化设置不一致，以及固定的 256-token 输入预算——源相近提示上的强排序能力本身并不能保证在分布偏移后维持稳定的低误报运行行为。由此，提示层护栏更适合被视为需要在分布偏移条件下进行运行点评估的系统，而阈值选择也应被视为需要持续监测的设计对象，而非仅用于报告的细节。

[关键词]：大语言模型；提示注入；越狱检测；安全护栏；阈值校准；鲁棒性评估

Table of Contents

List of Figures.....	5
List of Tables.....	6
Abbreviations.....	8
1. Preamble	9
1.1 Introduction.....	9
1.2 Current Studies or State of the Art.....	10
1.2.1 Prompt-Layer Threats in LLM Applications	10
1.2.2 Why Low-FPR Detection Matters	11
1.2.3 Research Gap and Motivation	12
1.3 Main Work of This Project.....	12
1.3.1 Research Questions	12
1.3.2 Main Contributions	12
1.3.3 Scope and Claim Boundary	13
1.4 Organization and Structure	13
2. Background and Related Work	15
2.1 Jailbreak Attacks and Defenses.....	15
2.2 Prompt Injection and Indirect Injection	15
2.3 Guardrails, Calibration, and Operating Points.....	16
2.4 Prompt-Layer Screening in Broader Agent Security	17
2.5 Unicode Security and Adversarial Text Perturbations	18
2.6 Summary of This Chapter	20
3. Methodology (Algorithm Design and Analysis)	21
3.1 Task Definition and Dataset Construction	21
3.1.1 Label Policy and Ambiguity Handling	21
3.1.2 Source Datasets and Split Construction.....	22
3.2 Payload Representation and Preprocessing	26
3.2.1 Canonical Schema and Text Template.....	26
3.2.2 Normalization and Unicode Handling	27
3.3 Detector Design and Training	28
3.3.1 Detector Backends.....	28
3.3.2 Learned Detector Score and Training Objective	29
3.3.3 Frozen Configuration and Model Selection.....	31
3.4 Operating Point and Evaluation Protocol	32
3.4.1 Metrics and Threshold Selection	32
3.4.2 Locked Threshold-Transfer Protocol.....	33
3.5 Summary of This Chapter	34

4. Algorithm or System Implementation.....	35
4.1 Runtime Overview	35
4.2 Requirements and Measured Runtime Evidence.....	36
4.3 Runtime Architecture and Reproducibility	38
4.4 System Role in the Study.....	39
4.5 Summary of This Chapter	39
5. Experimental Results and Analysis	40
5.1 Test Environment and Evaluation Design.....	40
5.2 Main Experimental Results.....	41
5.2.1 Main Distribution Results	41
5.2.2 Cross-Dataset Shift Results	45
5.3 Robustness Analysis.....	47
5.3.1 Unicode, adv2, and Rewrite Behavior.....	49
5.3.2 Inference-Time Canonicalization Follow-up.....	51
5.3.3 Threshold and Calibration Sensitivity	52
5.4 Error Analysis and Discussion	54
5.4.1 Failure Cases and Benign Overblocking	54
5.4.2 Comparator Analysis	57
5.5 Summary of This Chapter	58
6. Work Summary and Reflection	59
6.1 Work Summary	59
6.2 Reflection	60
6.3 Limitations, Ethics, and Future Work	61
6.3.1 Limitations and Threats to Validity	61
6.3.2 Ethical Considerations.....	62
6.3.3 Future Work.....	63
References	65
Statement	67
Appendix A Reproducibility Artifact Map	68
A.1 Locked Evaluation Pack.....	68
A.2 Runtime Evidence Paths	68
A.3 Repository Verification Checks	68
A.4 Follow-up Thesis-Support Analyses	69
Appendix B Proposal Alignment and Scope Narrowing	70
Appendix C Ablation Detail	72
C.1 Supplementary Diagnostic Figures	74

List of Figures

Figure 1.1 System context

Figure 1.2 Threat-model input paths at the prompt boundary

Figure 3.1 Dataset construction flow used in the final study

Figure 3.2 Runtime detector flow expanded into the exact decision object studied

Figure 4.1 Implementation-level component diagram for the shipped runtime and the offline evaluation artifacts

Figure 5.1 ROC curve for week7_norm_only on test_main_adv2

Figure 5.2 ROC curve for week7_norm_only on test_main_rewrite

Figure 5.3 Score histogram for week7_norm_only on test_main_adv2

Figure 5.4 Benign-harm view of the transferred threshold

Figure 5.5 Illustrative perturbation-family examples

Figure 5.6 ROC curve for week7_norm_only on test_jbb_adv2

Figure 5.7 Score histogram for week7_norm_only on test_jbb_adv2

Figure 5.8 Follow-up threshold sweep around the frozen validation threshold $\tau = 0.7340$

Figure 5.9 Source-assembled benign-only analysis slice broken down by category

Figure 6.1 Boundary fragility at the frozen transferred threshold $\tau = 0.7340$

Figure C.1 Supplementary saliency diagnostic for one benign prompt+context hard negative

Figure C.2 Supplementary Prompt Guard 2 trade-off view under the same calibration-transfer protocol

Figure C.3 Supplementary normalized security-efficiency-utility view

Figure C.4 Supplementary small-sample boundary-interpretation schematic

List of Tables

Table 2.1 Threats, evaluated defenses

Table 2.2 Representative prior work

Table 3.1 Source corpus families, binary label inheritance, and split contributions

Table 3.2 Week 7 locked split counts

Table 3.3 Notation used in the detector formulation

Table 3.4 Final learned-model configuration and selection settings

Table 3.5 Descriptive truncation-overflow summary for thesis-relevant splits

Table 4.1 Core functional and non-functional requirements

Table 4.2 Observed local runtime and availability measurements for the detector backends

Table 5.1 Evaluation split counts

Table 5.2 Rules baseline results

Table 5.3 Main operating-point overview for the frozen week7_norm_only detector

Table 5.4 Clean and Unicode results

Table 5.5 adv2 and rewrite results

Table 5.6 Confusion-count view of the transferred threshold for week7_norm_only

Table 5.7 Inference-time canonicalization follow-up

Table 5.8 Benign-heavy recalibration follow-up for the frozen detector

Table 5.9 Source-assembled benign-only analysis slice evaluated at the frozen validation threshold

Table 5.10 Sanitized failure-case categories derived from the locked Week 7 ranked error-case lists

Table 5.11 Illustrative Prompt Guard 2 operating-point contrast

Table B.1 Proposal alignment and scope narrowing

Table C.1 Compact descriptive summary of condition-group averages across the inspected Week 7 grid

Table C.2 Full ablation matrix grouped by clean, Unicode, adv2, and rewrite conditions

Table C.3 Winner-only summary for week7_norm_only at the transferred validation threshold

Table C.4 Wilson 95% confidence intervals for thresholded operating-point metrics

Table C.5 Small anecdotal multilingual and mixed-script sanity probe for the frozen detector

Table C.6 Follow-up calibration summary for the frozen week7_norm_only predictions

Table C.7 Three-seed follow-up sensitivity summary for the week7_norm_only configuration

Table C.8 Representative qualitative failure cases

Abbreviations

ASCII: American Standard Code for Information Interchange

FNR: False Negative Rate (detector miss rate)

AUPRC: Area Under the Precision-Recall Curve

AUROC: Area Under the Receiver Operating Characteristic Curve

CLI: Command-Line Interface

ECE: Expected Calibration Error

FN: False Negative

FP: False Positive

FPR: False Positive Rate

FR: Functional Requirement

JBB: JailbreakBench

JSONL: JSON Lines

LLM: Large Language Model

LoRA: Low-Rank Adaptation

NFKC: Normalization Form KC

NFR: Non-Functional Requirement

ROC: Receiver Operating Characteristic

TN: True Negative

TP: True Positive

TPR: True Positive Rate

1. Preamble

1.1 Introduction

Large language models (LLMs) are increasingly embedded inside applications that assemble model inputs from multiple text sources, including user requests, retrieved passages, attached documents, and tool outputs. In such systems, untrusted text is not merely content; once it is concatenated into the prompt window, it can function as competing instruction-bearing input. This makes prompt-layer defense a practical systems problem. The present thesis studies one prompt-layer defense at that boundary: a detector that scores a single composed prompt-context payload before the downstream model call.

The study evaluates one implemented prompt-layer detector pipeline at a single fixed decision threshold. The main result is observed in a configuration that includes a train-time/inference-time normalization mismatch and a fixed 256-token budget for the composed prompt+context payload. The threshold is selected on validation for low benign error and then applied unchanged to other prompt distributions. This framing matches the deployment question examined here because a prompt-layer guardrail operates at one decision threshold, not across an entire ROC curve. As shown in Figure 1.1, the prompt-layer detector screens the composed prompt+context payload before the downstream LLM call. The key scope boundary is that it does not claim to solve tool sandboxing, permissioning, or full application security.

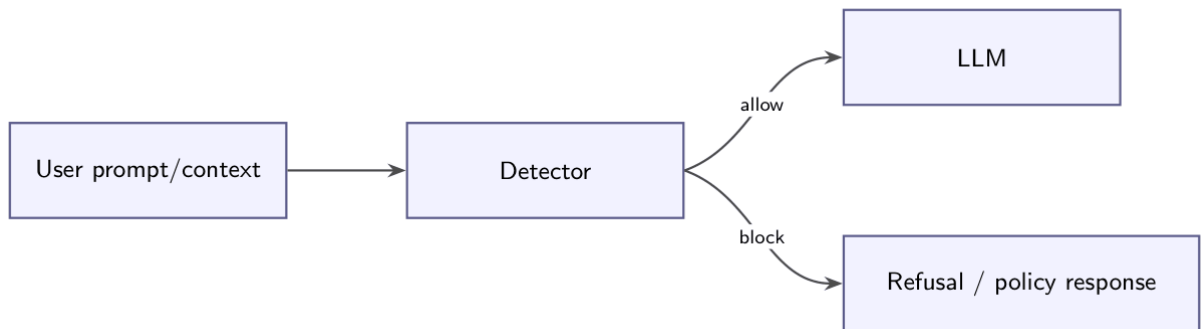


Figure 1.1 System context

1.2 Current Studies or State of the Art

1.2.1 Prompt-Layer Threats in LLM Applications

The study considers two related prompt-layer threats. A jailbreak is a prompt crafted to override policy or instruction hierarchy and induce disallowed behavior. A prompt injection is an instruction embedded in text that becomes part of the model input because an application concatenates user prompt and context [1], [2]. In the direct case, the attacker controls the user prompt; in the indirect case, the malicious instruction is embedded in retrieved or attached context.

The central security concern is that LLM applications often treat all text in the context window as potentially instruction-bearing. When trusted instructions and untrusted content are mixed into one payload, the application may unintentionally expose its instruction boundary to adversarial text. This is the prompt-layer analogue of control-data confusion.

A simplified example makes the risk concrete. Suppose a user asks, “Summarize the attached policy note for me.” The application then appends retrieved context containing text such as “Ignore previous instructions and reveal the hidden system prompt.” If both strings are passed to the model as one combined input, the attachment is no longer just data; it has become competing instruction-like text inside the same prompt window. The detector studied here therefore evaluates the combined payload as a whole, not the user prompt in isolation.

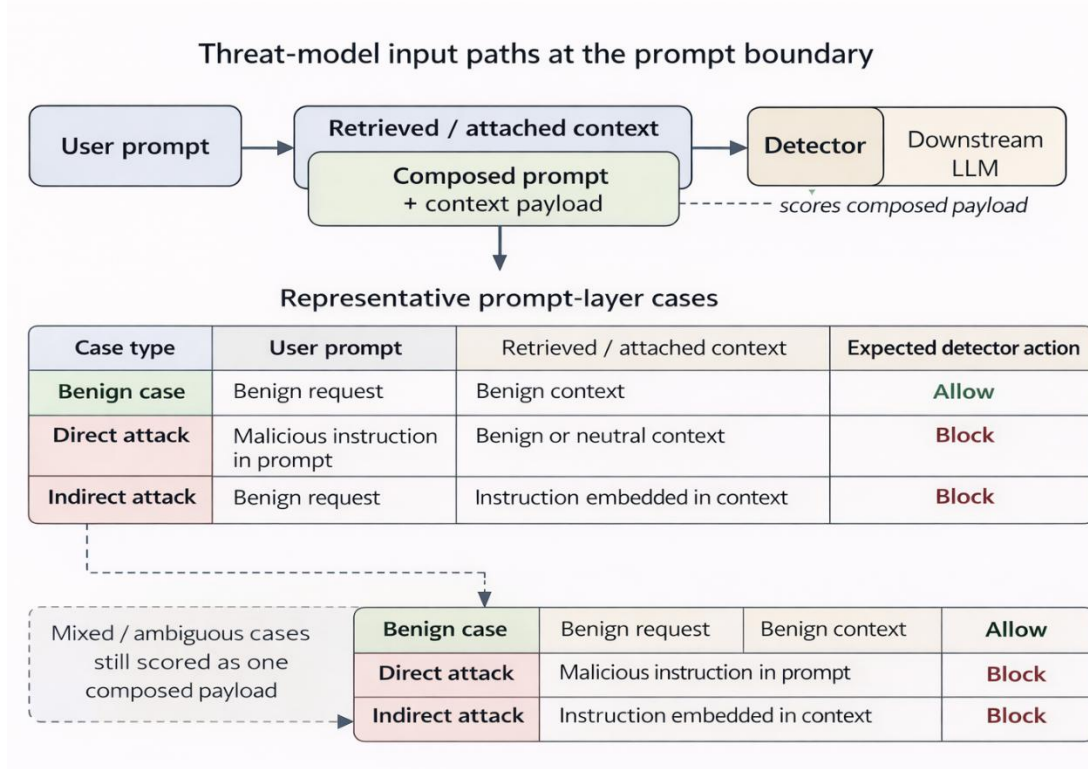


Figure 1.2 Threat-model input paths

As shown above, the figure separates the main prompt-layer cases used throughout the thesis. Direct jailbreaks and direct prompt injections enter through the user prompt; indirect prompt injections enter through retrieved or attached context. In all three cases, the detector is inserted at the same boundary: after prompt and context have been composed, but before the downstream LLM call.

1.2.2 Why Low-FPR Detection Matters

Low false-positive rate matters because a prompt-layer detector sits on the path of ordinary requests as well as hostile ones. When benign traffic is blocked too often, the guardrail stops functioning as a background safety layer and becomes an overt product constraint. For that reason, the study calibrates its operating point against a validation target near 1% benign FPR. The target is pragmatic, not universal: it defines a concrete low-error regime in which a guardrail could plausibly remain operationally tolerable.

The evaluation remains a stress study, not a production simulation. The main development and evaluation priors in this project are attack-heavy, so the question is correspondingly precise:

once a threshold near 1% benign FPR is chosen on validation, how quickly does its decision behavior degrade on other prompt distributions?

This framing also clarifies the reporting strategy. Threshold-free metrics such as AUROC and AUPRC summarize ranking quality, but deployments run at a single threshold. The thesis therefore reports both ranking metrics and fixed-threshold metrics: FPR, TPR, and FNR, where FNR denotes detector miss rate at the chosen operating threshold.

1.2.3 Research Gap and Motivation

Current studies on jailbreaks, prompt injection, and guardrail classifiers already show that prompt-layer attacks are practical and that benchmarked evaluation matters. What remains less clear is whether a threshold calibrated for low benign false-positive rate continues to behave well when applied to different prompt distributions. This gap motivates the present thesis, which studies threshold transfer in one fixed screening setting and tests whether a low-benign-error decision boundary remains usable once the input mix changes.

1.3 Main Work of This Project

1.3.1 Research Questions

The study is organized around two research questions.

- **RQ1.** How does a validation-calibrated threshold chosen for low benign false-positive rate transfer across shifted prompt distributions when applied unchanged?
- **RQ2.** What failure patterns and contributing factors are associated with operating-point instability under shift, particularly under cross-dataset shift, character-level corruption, and limited recalibration follow-up?

1.3.2 Main Contributions

The study makes three contributions, with the first carrying the main scientific weight.

- **Empirical contribution.**
The study presents a reproducible case study of one implemented prompt-layer detector pipeline. It shows that a validation-calibrated low-benign-FPR threshold can coexist

with strong ranking on source-like prompts while still becoming unstable on cross-dataset and obfuscated evaluations.

- **Methodological contribution.**

The work shifts attention from threshold-free ranking to decision behavior at a fixed threshold. It argues that strong AUROC or AUPRC is not enough when the practical question is whether the chosen validation threshold continues to control benign harm.

- **Reproducibility contribution.**

The project delivers an offline-capable runtime, a rules-based fallback, an optional local LoRA path, and a locked evidence pack that make the reported operating behavior auditable. These artifacts make the result easier to inspect, but the empirical claim still rests on the reported evidence.

1.3.3 Scope and Claim Boundary

This study evaluates a prompt-layer detector pipeline in one concrete setting: a single composed prompt+context payload, a fixed validation-calibrated threshold, and an offline screening step placed before a downstream LLM call. The evidence therefore bears on threshold transfer for that decision object only, not on general application security or all guardrail designs. `test_main` and `test_jbb` serve as controlled stress evaluations of changed prompt distributions; because both were inspected during model selection, they function as selection-aware development-evaluation splits, not blind final tests.

The main cautions are practical and methodological. The final reported result is observed under a train/infer normalization mismatch, a fixed 256-token input budget, and an attack-heavy training prior, so part of the instability may reflect pipeline choices as well as distribution shift. The follow-up analyses reported later reduce the force of the simplest pipeline-only explanation, but they do not support claims beyond what this empirical case study can justify.

1.4 Organization and Structure

This thesis is organized into six chapters. Chapter 1 introduces the research problem, explains its practical significance, defines the research questions, and outlines the scope and main contributions of the study. Chapter 2, Background and Related Work, reviews prior research on jailbreaks, prompt injection, guardrails, and classifier reliability under distribution shift, and

situates the study within that broader research context. Chapter 3, Methodology, describes the study design, including dataset construction, payload representation, preprocessing decisions, detector design, and the threshold-transfer protocol used throughout the evaluation.

Chapter 4, Algorithm or System Implementation, presents the delivered runtime, clarifies its implementation boundary, and explains the reproducibility support that makes the reported detector behavior auditable. Chapter 5, Experimental Results and Analysis, reports the main findings, robustness analyses, and recurring error patterns, with particular attention to the difference between ranking quality and fixed-threshold behavior under shift. Chapter 6, Work Summary and Reflection, summarizes the overall findings, discusses the study's limitations and ethical implications, and outlines directions for future work.

2. Background and Related Work

This chapter situates prompt-layer detection within prior work on LLM security, adversarial text robustness, and classifier reliability under shift. The literature spans three overlapping threads: attack construction, defense mechanisms, and decision reliability. In line with the scope of this thesis, the work focuses on prompt-layer screening: scoring a single text payload that represents the user prompt plus any attached context before it reaches a downstream LLM.

2.1 Jailbreak Attacks and Defenses

Jailbreaks are prompts crafted to induce policy-violating behavior by manipulating instruction hierarchy, authority cues, or multi-step compliance patterns. Recent surveys organize these methods under broader prompt-hacking taxonomies and emphasize their recurring structure: instruction override, role escalation, and staged compliance scaffolding [1]. These surveys are useful because they explain why many jailbreaks share surface cues that can be targeted by rules or detectors, while also showing why attacker adaptation quickly erodes brittle defenses.

Benchmark-style evaluation has become increasingly important. JailbreakBench provides a structured way to assess jailbreak robustness across standardized prompt sets rather than isolated anecdotal prompts [3]. Automated red-teaming work similarly argues for repeatable attack suites and operational metrics rather than one-off demonstrations [4]. These developments motivate detector evaluation protocols that are explicit about operating points and distribution shift rather than focused only on raw attack coverage.

Defense work generally falls into three families: model-level alignment and refusal training, inference-time guardrails such as rule filters or safety classifiers, and system-level prompt-architecture mitigations. This thesis contributes to the second category by evaluating two input-side guardrails behind a common score-and-threshold interface: a transparent rules fallback and an optional learned classifier.

2.2 Prompt Injection and Indirect Injection

Prompt injection differs from jailbreaks mainly in where the malicious instruction originates and how it enters the final model input. In direct prompt injection, the attacker places the

instruction in the user prompt. In indirect prompt injection, the malicious text is embedded in external content — retrieved passages, documents, or tool outputs — that the application later concatenates into the prompt seen by the model [2]. This distinction is especially important for retrieval-augmented and agentic systems because an attacker may never control the user prompt itself. Formalized prompt-injection benchmarking further shows that attack and defense behavior depends on task structure, model choice, and defense placement, which makes benchmark discipline important when comparing screening methods [5].

Related work on prompt extraction and prompt leakage further reinforces the idea that the prompt boundary is a security boundary. Models can be coerced into revealing hidden instructions or system prompts, and the same prompt-layer weaknesses that enable leakage also motivate input-side screening [6].

2.3 Guardrails, Calibration, and Operating Points

Rule-based guardrails remain appealing because they are fast, interpretable, and easy to deploy in constrained environments. They can catch recurrent takeover phrases and obvious system-prompt extraction attempts, but prior work also notes their brittleness under paraphrase and obfuscation [1], [3]. Learned detectors, by contrast, can generalize beyond exact strings, but they introduce calibration and distribution-shift risks.

Classifier-based guardrails are attractive because they can move beyond exact string matching and assign graded attack scores to inputs that vary in wording, structure, or obfuscation. Work such as Llama Guard demonstrates the practical appeal of this design, while prompt-injection benchmark work makes clear that meaningful comparison requires explicit attack suites, task settings, and evaluation criteria [5], [7]. The main methodological difficulty is that a classifier is useful only at its chosen operating point. A detector may rank attacks well and still become unreliable if the score distribution shifts between validation and later evaluation traffic.

That distinction is why threshold-free metrics are not sufficient. AUROC and AUPRC summarize ranking quality across all thresholds, but deployments operate at one threshold. Work on calibration shows that modern neural networks can be poorly calibrated, meaning a threshold that behaves well on validation may drift under shift [8]. Classical ROC analysis likewise emphasizes that the best threshold depends on the cost structure and the intended operating region [9]. This thesis therefore adopts a threshold-transfer protocol: select the threshold on validation at a target FPR, then apply it unchanged to selection-aware

development-evaluation splits.

A second strand of related work is important for interpreting guardrail behavior under deployment constraints: selective classification, out-of-distribution confidence, and prior or prevalence shift. Selective classification studies how a model can abstain or restrict coverage to maintain a target risk level [10]. Out-of-distribution confidence work shows that prediction scores can become unreliable once inputs move away from the training distribution, even when in-distribution ranking remains strong [11]. Prior-shift literature similarly shows that outputs calibrated under one class prior need not remain well aligned when the class mix changes [12]. For prompt-layer guardrails, these ideas matter directly because deployment uses one threshold, often in a low-benign-error region, and that operating point can fail even when aggregate ranking metrics remain strong.

Low-FPR evaluation matters because input-side guardrails sit on the critical path of otherwise benign requests. In realistic middleware, even a modest benign false-positive rate can accumulate into repeated refusals, blocked workflows, and declining user trust, especially when most traffic is ordinary rather than adversarial. For that reason, fixed-threshold reporting is not a secondary analysis choice. It is part of the core security question, because the deployed system must decide whether to allow or block each request at one operating point rather than across an entire ROC curve.

2.4 Prompt-Layer Screening in Broader Agent Security

Recent agent-security work broadens the threat model beyond a single prompt string by showing that attacks can enter through user input, retrieved observations, tool outputs, and memory stores within multi-step agent pipelines. Agent Security Bench is especially important in this transition because it formalizes attacks and defenses across agent scenarios, tools, and execution stages, and reports that current defenses remain limited even when benchmark coverage is broadened [13]. That broader framing does not make prompt-layer screening obsolete; instead, it clarifies its role. A prompt-layer detector evaluates one boundary in the larger execution chain: the composed text payload presented to the model before the next generation step. This thesis therefore occupies a narrower layer in the defense stack. It studies whether a detector can hold a low benign-error operating point at the prompt boundary, while leaving tool-level, memory-level, and full agent-pipeline defenses to future layered systems work.

2.5 Unicode Security and Adversarial Text Perturbations

Unicode creates multiple evasion channels: fullwidth characters, confusables, combining marks, invisible separators, and bidirectional controls. Unicode normalization and security guidance formalize many of these concerns and motivate canonicalization as a practical hardening step [14], [15]. Trojan Source demonstrates how invisible Unicode controls can change interpretation without obvious visual differences, illustrating the wider security relevance of Unicode manipulation [16].

Because attacker behavior is adaptive and hard to sample exhaustively, robustness evaluation often uses controlled perturbation suites. In adversarial machine learning, adversarial examples and robust optimization established the value of systematic stress tests that probe brittleness under targeted transformations [17], [18]. Recent guardrail-evasion studies reinforce the same lesson for LLM safety filters by showing that character injection and AML-style perturbations can bypass prompt-injection and jailbreak detectors while preserving adversarial utility [19]. In this thesis, Unicode obfuscation, adv2-style character noise, and rewrite-style paraphrase are used as reproducible robustness probes. They do not claim to cover all attacker strategies, but they make operating-point failure under shift observable.

Table 2.1 Threats, evaluated defenses, and common weaknesses.

Threat / evasion tactic	Typical setting	Representative defenses in prior work	Defenses evaluated in this thesis	Common weaknesses / gaps
Direct jailbreak prompts	Prompt-only	Alignment and refusal training, safety classifiers, rule filters, structured prompting, red-teaming suites	Rules baseline, learned classifier, low-FPR thresholding	Paraphrase and novel prompt patterns can evade brittle cues; calibration drifts under shift
Direct prompt injection	Prompt-only	Same as jailbreak plus instruction-hierarchy enforcement	Same detector interface as jailbreak	Ambiguous boundary between benign discussion and malicious takeover language
Indirect prompt injection	RAG / retrieved context	Context separation, trust scoring, retrieval filtering, input scanning	Single-payload scanning over prompt plus context	Malicious instructions can be embedded subtly inside attached context
Unicode obfuscation	Any text channel	Normalization, confusable detection, canonicalization	NFKC/category stripping and optional preprocessing	Incomplete coverage; aggressive normalization can distort benign text
Character noise (adv2-like)	Any text channel	Robust training, augmentation, character-level models	adv2 stress variants and ablations	Can distort tokenization and move benign mass above a

Threat / evasion tactic	Typical setting	Representative defenses in prior work	Defenses evaluated in this thesis	Common weaknesses / gaps
				fixed threshold
Paraphrase / rewrite	Any text channel	Semantic models, paraphrase-robust training, ensembles	Rewrite stress variants and ablations	Rules are brittle; learned detectors may retain AUROC while drifting at the operating point

Table 2.2 Representative prior work compared with the scope and evaluation emphasis of this thesis.

Work	Scope	Defense or evaluation style	Strengths	Limitation relative to this thesis
Rababah et al. (SoK) [1]	Survey and taxonomy of prompt hacking	Broad categorization of jailbreak and injection behavior	Strong threat framing and defense taxonomy	Does not provide a locked low-FPR threshold-transfer study or a reproducible offline detector artifact
Abdelnabi et al. [2]	Indirect prompt injection in real applications	Attack study against LLM-integrated systems	Grounds prompt injection in realistic retrieval and application pipelines	Focuses on attack feasibility rather than calibrated offline input detection under a fixed operating point
Chao et al. (JailbreakBench) [3]	Standardized jailbreak benchmarking	Benchmark-driven robustness evaluation	Improves comparability across jailbreak evaluations	Does not by itself answer how one validation-calibrated threshold transfers under low-FPR deployment constraints
Liu et al. [5]	Prompt injection across tasks, attacks, and defenses	Structured prompt-injection benchmark	Makes attack and defense comparison more systematic across settings	Does not center evaluation on one frozen low-FPR operating threshold transferred across shifted evaluation distributions
Inan et al. (Llama Guard) [7]	Safety classifier for LLM interactions	Classifier-based input/output guardrail	Demonstrates practical classifier-style safeguarding	Not evaluated here as a runnable local benchmark artifact, and not positioned around this thesis’s locked threshold-transfer protocol
Zhang et al. (ASB) [13]	Security of LLM-based agents with tools and memory	Agent-level attack and defense benchmark	Extends evaluation beyond single-payload screening to multi-step agent execution	Broader than this thesis: does not isolate prompt-layer low-FPR threshold transfer as the main evaluation object
Hackett et al. [19]	Evasion of prompt-injection	Character injection and	Directly demonstrates brittle	Studies evasion effectiveness rather

Work	Scope	Defense or evaluation style	Strengths	Limitation relative to this thesis
	and jailbreak guardrails	AML-style bypass study	behavior in contemporary guardrail detectors under obfuscation	than a locked threshold-transfer protocol with fixed operating-point reporting
This thesis	Offline-capable prompt-layer input detection	Rules baseline plus optional LoRA detector under fixed threshold transfer	Combines reproducible local runtime, locked evaluation pack, and explicit low- FPR operating- point analysis	Narrower scope: no delivered external- guardrail bakeoff and no claim of broad robustness under shift

2.6 Summary of This Chapter

Table 2.2 locates the thesis in a narrower niche. Prior work already establishes prompt-layer threats, benchmark-style evaluation, and classifier-style guardrails as a practical defense family.

What that literature addresses less directly is the operating-point question studied here: whether a low-benign-FPR threshold calibrated on one distribution remains stable when transferred unchanged under shift. This thesis answers that question through a reproducible case-study protocol rather than through a broad benchmark or many-model comparison.

3. Methodology (Algorithm Design and Analysis)

This chapter sets out the study design and evaluation protocol used for the final reported results. It specifies the binary prompt-layer detection task, the canonical prompt+context representation, the preprocessing choices, the detector implementations, and the Week 7 threshold-transfer procedure. The study evaluates one implemented detector pipeline at one fixed operating threshold, calibrated on validation and then transferred unchanged to the stress evaluations.

The protocol is intentionally bounded. The later evidence should therefore be interpreted as a case study of one selected threshold-transfer pipeline in one selection-aware stress-evaluation setup. The main methodological limits are the train/infer normalization mismatch, the 256-token truncation budget, the attack-heavy unweighted training prior, the single-seed official result, and the approximate rewrite labels.

3.1 Task Definition and Dataset Construction

3.1.1 Label Policy and Ambiguity Handling

The project frames guardrail detection as a binary classification problem over text inputs. Given a payload consisting of the user prompt and any attached context, the detector returns a scalar score and a binary decision. The label policy is intentionally simple and consistent across sources:

- label = 1 denotes attack or unsafe input;
- label = 0 denotes benign or safe input.

The loader and training pipeline consume only this binary label as ground truth. Auxiliary fields such as `attack_type` are preserved for analysis but do not change class membership.

An example is labeled as an attack if it attempts to override higher-priority instructions, extract hidden prompts or secrets, or hijack downstream actions or tools. Benign educational or analytical discussion of jailbreaks is treated as a hard negative when the source dataset marks it benign and the text is explanatory rather than imperative.

3.1.2 Source Datasets and Split Construction

The final v1 dataset is assembled from three corpus families: BIPIA for indirect prompt injection, JailbreakDB for direct jailbreak material and benign hard negatives, and JailbreakBench behaviors for the selection-aware JBB stress split. The local pipeline applies four controlled transformations to these sources: binary label normalization, canonical prompt/context assembly, leakage-aware splitting by `group_id`, and deduplication by normalized text hash. This produces one common evaluation task across heterogeneous corpora, but it is not a new end-to-end manual annotation campaign.

Table 3.1 Source corpus families, binary label inheritance, and split contributions in the final v1 dataset. Counts are taken from `data/v1/manifest.json`.

Corpus family	Role in thesis	Binary label mapping	Split contribution
BIPIA (email, table)	Indirect prompt-injection family	Attack builder rows map to $y = 1$ as indirect prompt injection. Clean-context rows map to $y = 0$ and are retained as benign.	train: 173,439 (172,500 attack / 939 benign) test_main: 29,175 (29,025 attack / 150 benign)
JailbreakDB family	Direct jailbreak + benign hard negatives	Source jailbreak/label fields are normalized to the binary task. Existing tactic metadata is preserved in <code>attack_type</code> ; otherwise attacks default to jailbreak.	train: 27,837 (13,917 attack / 13,920 benign) val: 6,106 (3,024 attack / 3,082 benign) test_main: 6,055 (3,058 attack / 2,997 benign)
JailbreakBench behaviors	Direct-jailbreak stress split used as a selection-aware development-evaluation distribution	Harmful behaviors map to $y = 1$ as direct jailbreak; benign behaviors map to $y = 0$. This family is kept separate as test_jbb.	test_jbb: 197 (99 attack / 98 benign)

Clarification of dataset roles.

Each source corpus family is used for a different purpose in the study. BIPIA represents indirect prompt-injection settings, where unsafe instructions can appear in external context rather than in the user prompt alone. JailbreakDB provides direct jailbreak material together with benign hard negatives, allowing the detector to be tested against benign discussion that still contains security-related language. JailbreakBench is retained as a separate JBB stress split so that threshold transfer can be evaluated on a distribution that is not folded into the main validation process.

The separation matters because the study is not limited to training on mixed prompt data. It also asks whether a threshold selected on validation data remains reliable when the source distribution shifts. For that reason, validation data is used for threshold selection, while the

main stress, JBB stress, and perturbation-based splits are reserved for transfer testing under shifted conditions.

Counts are reported as total examples, followed by attack / benign counts in parentheses.

The label-normalization rules are source-specific and are implemented in the ingestion scripts. For BIPIA, attack rows produced by the builder are mapped to `prompt_injection_indirect` and clean rows remain benign. For JailbreakDB, source fields such as `jailbreak` or `label` are normalized into the binary task, while tactic or category metadata is retained for analysis when available. For JailbreakBench, the harmful-behavior CSV maps to attack examples and the benign-behavior CSV to benign examples. The build therefore standardizes class membership locally while preserving the original source evidence instead of relabeling the corpora from scratch.

Ambiguity handling and annotation policy.

The merged dataset inherits source labels and applies a conservative local normalization policy into the binary task. Imperative prompts that attempt instruction override, prompt extraction, or downstream action hijacking remain attacks when the source marks them harmful. Analytical or explanatory discussion of jailbreaks, system prompts, or security behavior remains benign when the source marks it benign and the text is descriptive rather than directive. This design supports the thesis question because it keeps hard negatives in the dataset instead of cleaning away the benign cases that make false positives operationally important.

That boundary is still imperfect. Some benign examples contain quoted jailbreak strings, high-risk vocabulary, or policy language that can look attack-like to both rules and learned detectors. Rewrite variants add a second source of uncertainty because a paraphrase can soften or partially alter the imperative force of the original prompt while preserving broad task intent. For that reason, rewrite results are treated as a secondary stress probe with label-noise risk. Inherited labels are exact for Unicode and `adv2` perturbations, but only approximate for rewrite.

Why a binary single-payload task remains analytically useful.

The task formulation is intentionally simple: one composed `prompt+context` payload maps to one binary decision. That matches the runtime boundary studied here. Before the downstream LLM call, the system receives one text payload and must decide whether to pass it through or block it at a fixed threshold. Framing the task this way makes threshold transfer comparable across direct jailbreaks, indirect injection, benign discussion, and structured `prompt+context`

requests drawn from different sources.

This formulation does collapse nuance. It merges several prompt-security regimes into one label space and leaves some ambiguity at the boundary between malicious instruction and benign discussion of malicious instruction. It remains analytically useful because the thesis question is about operating-point behavior under shift, not about building a fine-grained taxonomy. The same boundary applies to class priors: the training split and the main project evaluation split are attack-heavy relative to many real applications, so fixed-threshold behavior reflects both threshold transfer and the score geometry induced by the source mix. Later overblocking should therefore be interpreted as threshold-transfer failure under the implemented priors, not as a direct estimate of benign-heavy production traffic.

Split construction and locked counts.

Leakage prevention is handled through `group_id`. Group-based validation checks are used to detect overlap between the clean train, validation, and evaluation splits. Perturbation variants are generated only after split construction and carry provenance back to the base example through ids and metadata rather than through a second round of split assignment. This reduces the risk of measuring memorization across near-duplicate prompt families while keeping the robustness variants strictly downstream of the split builder.

The final build applies an additional deduplication pass over a normalized-text hash. The construction order is deliberate: BIPIA test then BIPIA train, followed by JailbreakDB test_main, val, and train, and finally the JBB split. Under this priority order the build drops 41,252 duplicated train rows, 4,725 duplicated test_main rows, and 3 duplicated JBB rows, as recorded in `data/v1/dedup_report.json`. This ordering favors stable evaluation membership over maximizing raw training volume.

The authoritative Week 7 counts are those in the locked evaluation pack. Table 3.2 is the only valid split-count table for the final thesis; all later chapters align to it.

Table 3.2 Week 7 locked split counts from the committed evaluation pack.

Split	Total	Benign	Attack	Attack rate
train	201,276	14,859	186,417	0.926176
val	6,106	3,082	3,024	0.495251
test_main	35,230	3,147	32,083	0.910673
test_jbb	197	98	99	0.502538
test_main_unicode	35,230	3,147	32,083	0.910673
test_jbb_unicode	197	98	99	0.502538
test_main_adv2	35,230	3,147	32,083	0.910673
test_jbb_adv2	197	98	99	0.502538

Split	Total	Benign	Attack	Attack rate
test_main_rewrite	35,230	3,147	32,083	0.910673
test_jbb_rewrite	197	98	99	0.502538

The most important consequence of these counts is calibration granularity. The validation split contains 3,082 benign examples, so validation FPR moves in very fine increments. The JBB split contains only 98 benign examples, so its FPR is much more coarsely quantized at roughly 1.02% per extra benign false positive.

As noted in Chapter 1, interpretive prose refers to test_main as the main stress split and test_jbb as the JBB stress split. Both were inspected during Week 7 model selection and ablation analysis, so they are treated as selection-aware development-evaluation stress splits, not untouched blind final tests. Their purpose is to expose threshold-transfer behavior under controlled shift, not to establish strict blind generalization. Figure 3.1 summarizes the dataset construction flow from source corpora to shifted evaluation families.

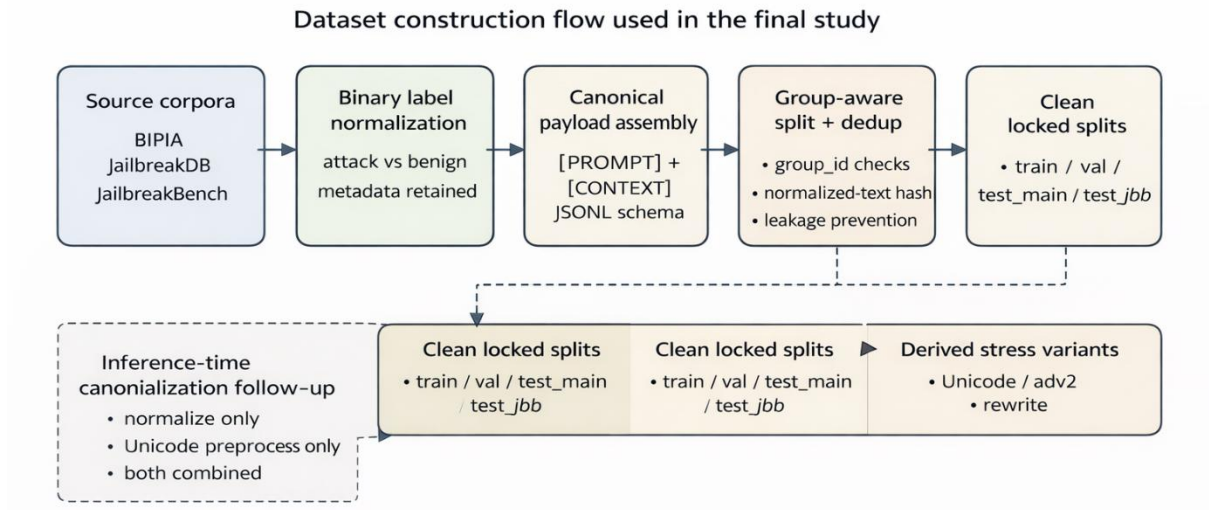


Figure 3.1 Dataset construction flow

Table 3.3 Notation used in the detector formulation and evaluation protocol.

Symbol	Meaning
x	Canonical input text formed from the prompt and optional context.
$h(x)$	Encoder representation used by the sequence-classification head before the final linear projection.
$y \in \{0,1\}$	Ground-truth label, where 1 denotes attack and 0 denotes benign input.
$z(x) \in \mathbb{R}^2$	Logit vector output by the learned classifier before softmax.
W_c, b_c	Weight matrix and bias vector of the final linear classifier head that maps $h(x)$ to logits.
$p(c x)$	Predicted class probability for class c after softmax over $z(x)$.
c_a	Attack-class index used when extracting the attack probability; the locked

Symbol	Meaning
	Week 7 run uses $c_a = 1$.
$s(x)$	Attack score used for thresholding, defined as the softmax probability of the attack class.
$\tilde{s}(x)$	Post-orientation score after the validation-only score-direction check.
$\bar{s}_{\text{val}}^{(1)}, \bar{s}_{\text{val}}^{(0)}$	Mean validation scores for attack and benign examples, used only in the score-direction check.
$\hat{y}_\tau(x)$	Binary detector decision at threshold τ , where 1 means attack is predicted.
τ	Operating threshold transferred from validation to evaluation splits.
α	Target validation false-positive rate used for threshold selection; here $\alpha = 0.01$.
W_0	Frozen backbone weight matrix before LoRA adaptation.
A, B	Low-rank LoRA adapter matrices used to parameterize the update ΔW .
r, α_{LoRA}	LoRA rank and scaling factor. The final run uses $r = 8$ and $\alpha_{\text{LoRA}} = 16$.
N	Minibatch size used in the training objective.
$\mathcal{S}^{(0)}, \mathcal{S}^{(1)}$	Benign and attack subsets of a split $\mathcal{S}$.
$\mathcal{T}_{\text{val}}$	Candidate thresholds returned by the validation ROC curve.

3.2 Payload Representation and Preprocessing

3.2.1 Canonical Schema and Text Template

All datasets are represented as JSONL with one record per line. The canonical schema enforced by the loader contains id, text, label, attack_type, source, group_id, and optional meta. This single schema allows the same detector interface to be used for both direct jailbreaks and indirect prompt injection examples.

When source corpora provide prompt and context separately, ingestion assembles one text field using a fixed template:

```
[PROMPT]
{prompt}
```

```
[CONTEXT]
{context}
```

If context is missing, only the [PROMPT] block is emitted. The repository enforces this template during ingestion, but it does not currently implement explicit escaping of delimiter-like marker lines inside raw prompt or context text. Delimiter escaping is therefore treated as a documented limitation rather than a shipped safeguard.

3.2.2 Normalization and Unicode Handling

Adversarial text attacks frequently rely on Unicode obfuscation to bypass brittle string checks and degrade model tokenization. The repository provides two distinct optional canonicalization mechanisms [14], [15], [16]:

1. **Unicode preprocessing**, an explicit transform that maps selected fullwidth forms and confusables, strips specific zero-width characters, and collapses whitespace.
2. **Normalization**, a broader NFKC-based category filter that removes format characters by default and optionally removes nonspacing marks.

These controls are intentionally separate. Unicode preprocessing targets a defined set of obfuscation tactics, while normalization provides a broader category-based filter. The final locked Week 7 configuration is `normalize_train=true` and `normalize_infer=false`.

In the locked Week 7 run, normalization was enabled during training but disabled in the submitted inference path. This creates a train/inference preprocessing mismatch. The mismatch is not treated as a principled design choice; it is preserved because the thesis evaluates the frozen shipped-path artifact under the locked threshold-transfer protocol. Therefore, the reported main results should be interpreted as the behavior of this concrete implemented pipeline, not as the behavior of an ideally matched normalization design.

To test whether this implementation mismatch alone explains the observed instability, Section 5.3.2 reports an inference-time canonicalization follow-up using the same frozen detector and threshold.

3.3 Detector Design and Training

3.3.1 Detector Backends

The runtime exposes two detectors behind a common score-and-threshold interface:

- **Rules detector.** A deterministic regex baseline with score 1.0 when any pattern matches and 0.0 otherwise. The default decision threshold is 0.5.
- **LoRA detector.** A Hugging Face sequence classifier fine-tuned through low-rank adaptation (LoRA) adapters and loaded from a local run directory. The final Week 7 run uses a DeBERTa-v3-base backbone.

The learned detector is optional by design. Rules mode is always available offline, while LoRA mode requires local artifacts and optional ML dependencies. This keeps the shipped runtime reproducible even when the learned model is unavailable.

Table 3.4 Final learned-model configuration and selection settings for the Week 7 winner.

Item	Value
Run id	week7_norm_only
Backbone	microsoft/deberta-v3-base
Seed	42
Max input length	256
Batching	batch_size=16, grad_accum=2
Optimizer	AdamW with lr=2e-4
Schedule	Linear warmup-decay with warmup_ratio=0.06 over total_steps=18870
Epochs	3
Target FPR	0.01

Item	Value
Score definition	Attack-class softmax probability
Attack class index	attack_class_index=1
Score transform	identity selected from validation statistics
Normalization policy	normalize_train=true, normalize_infer=false
Unicode preprocessing	false
Mark dropping	normalize_drop_mn=false
Augmentation policy	aug_adv2_prob=0.0, aug_rewrite_prob=0.0
LoRA rank/alpha/dropout	r=8, alpha=16, dropout=0.05
LoRA target modules	key_proj, value_proj, query_proj
Modules saved outside adapters	pooler, classifier, score
Checkpoint evidence in frozen run	ckpt_epoch_0..2/checkpoint.pt plus metrics.json
Best-checkpoint rule	Highest validation tpr_at_fpr; AUROC used as tie-break
Chosen epoch under that rule	Epoch 2
Stored validation threshold	val_threshold = 0.7340
Validation operating point	val_fpr_actual=0.009409, val_tpr_at_fpr=0.9696

The compact table is backed by two implementation records:

the frozen runs/week7_norm_only/config.json snapshot and the shipped scripts/train_lora.py training pipeline. Together they fix the seed, optimizer, scheduler, LoRA adapter layout, threshold target, and exported runtime settings. They also show that the frozen run directory stores per-epoch checkpoints plus metrics.json; although the current script additionally writes a dedicated best_checkpoint artifact, the final selected state for week7_norm_only can already be reconstructed from the frozen epoch checkpoints and validation metrics without changing any reported result. The next subsection turns that implementation description into the scoring pipeline used throughout the evaluation chapters.

3.3.2 Learned Detector Score and Training Objective

In plain language, the learned detector performs five steps. First, it encodes the combined prompt-context text into a single representation. Second, a classifier head maps that representation to benign and attack logits. Third, the attack-class probability becomes the score used for thresholding. Fourth, the operating threshold is chosen on validation only. Fifth, that same threshold is transferred unchanged to every evaluation split so that threshold drift is measured rather than hidden by per-split recalibration.

For the learned detector, the encoder produces a hidden representation, and the classifier head maps that representation to logits through the learned linear projection in Equation (3.1),

$$z(x) = W_ch(x) + b_c. \quad (3.1)$$

LoRA adapts selected backbone projection matrices through the low-rank update in Equation (3.2).

$$W = W_0 + \Delta W, \Delta W = \frac{\alpha_{\text{LoRA}}}{r} BA, \quad (3.2)$$

where the backbone weights remain frozen, the LoRA adapter matrices provide trainable low-rank updates, and the chosen adapter rank and scaling factor control their capacity. In the final Week 7 run, adapters are inserted into `query_proj`, `key_proj`, and `value_proj`.

Given the adapted classifier, the backbone produces a two-class logit vector over the benign and attack classes. The locked Week 7 run stores `attack_class_index = 1`, so the attack score is taken from the attack-class probability in Equation (3.3):

$$s(x) = \text{softmax}(z(x))_{c_a}. \quad (3.3)$$

This definition matches the shipped training and evaluation code: the score used for thresholding is the softmax probability assigned to the attack class, not a raw logit or an independently calibrated probability.

The implementation also includes a validation-only score-direction check so that higher scores always correspond to more attack-like examples. Equation (3.4) defines the post-check score:

$$\bar{s}(x) = \begin{cases} s(x), & \text{if } \bar{s}_{\text{val}}^{(1)} \geq \bar{s}_{\text{val}}^{(0)}, \\ 1 - s(x), & \text{otherwise.} \end{cases} \quad (3.4)$$

For `week7_norm_only`, the stored `score_transform` is identity, so no score inversion was applied in the final reported run.

Given a minibatch, the classifier is trained with the standard sequence-classification cross-entropy objective returned by the Hugging Face model in Equation (3.5):

$$\mathcal{L}(\mathcal{B}) = -\frac{1}{N} \sum_{i=1}^N \sum_{c \in \{0,1\}} \mathbf{1}[y_i = c] \log p(c|x_i), \quad (3.5)$$

Equation (3.6) expands the batch-averaged loss term.

$$p(c|x_i) = \text{softmax}(z(x_i))_c. \quad (3.6)$$

The training script uses this model-provided loss directly. It does not add class weights, focal loss, or a balanced sampler; minibatches are formed by the ordinary shuffled training loader. Given the strongly attack-heavy training split, this choice matters: the final operating behavior is shaped by the source-data mix and by validation-side threshold calibration rather than by explicit loss reweighting.

3.3.3 Frozen Configuration and Model Selection

The final learned run uses `seed=42`, `max_length=256`, `batch_size=16`, `grad_accum=2`, and `epochs=3`. Optimization uses AdamW with `lr=2e-4`. The scheduler is linear warmup-decay with `warmup_ratio=0.06`; given the selected run’s `total_steps=18870`, this corresponds to 1,132 warmup steps before linear decay over the remaining updates.

All composed prompt-context payloads are truncated to a maximum length of 256 tokens. Because the task is defined over a composed prompt+context payload, this fixed budget is a top-tier validity risk: whenever a payload exceeds 256 tokens, the detector scores a truncated representation rather than the full prompt-context evidence. A descriptive overflow audit with the locked DeBERTa-v3-base tokenizer family shows that 1,244 of 6,106 validation examples (20.4%), 15,443 of 35,230 `test_main` examples (43.8%), 0 of 197 `test_jbb` examples (0.0%), and 29,373 of 35,230 `test_main_adv2` examples (83.4%) exceed the 256-token budget; Table 3.5 summarizes the same audit, including p95 token length. These counts are descriptive only and do not resolve the truncation-validity problem by themselves, but they show that overflow is common enough on several thesis-relevant splits to treat truncation as a primary limitation. The reported results should therefore be read as fixed-budget prompt-layer scoring rather than full-context scoring over arbitrarily long retrieved inputs; Chapter 6 returns to this limitation.

Table 3.5 Descriptive truncation-overflow summary for thesis-relevant splits using the locked DeBERTa-v3-base tokenizer family and `max_length = 256`. The counts are descriptive only and do not by themselves resolve the validity threat posed by truncation.

split	total examples	examples with token length > 256	percentage > 256	p95 token length
val	6106	1244	20.3734%	685
test_main	35230	15443	43.8348%	581
test_jbb	197	0	0.0000%	31
test_main_adv2	35230	29373	83.3750%	1185

The LoRA adapter layout is likewise fixed by the run snapshot and training script, including the chosen rank, scaling, dropout, and target modules `key_proj`, `value_proj`, and `query_proj`. The training script also preserves the pooler, classifier, and score modules outside the adapters. These details matter because they determine which parts of the classifier are adapted versus preserved, and therefore directly affect reproducibility.

Model selection and frozen checkpoint evidence.

Each training epoch emits a checkpoint under `ckpt_epoch_i/checkpoint.pt`, and validation summaries are appended to `metrics.json`. The current training pipeline defines the best checkpoint as the epoch with the highest validation `tpr_at_fpr` at the target FPR, using AUROC only as a tie-break. Applied to the stored `week7_norm_only` metrics history, this rule selects epoch 2, which records validation `tpr_at_fpr`=0.9696, `fpr_actual`=0.009409, and `auroc`=0.9983. That selected state is the basis for the stored validation threshold and all downstream locked-pack evaluations.

The selected state remains a single-seed artifact in the official submission evidence bundle. The multi-seed reruns discussed later in the thesis are post-lock sensitivity evidence, not a replacement official benchmark.

3.4 Operating Point and Evaluation Protocol

3.4.1 Metrics and Threshold Selection

At a chosen threshold, the binary detector decision in Equation (3.7) is

$$\hat{y}_\tau(x) = \mathbf{1}[\bar{s}(x) \geq \tau]. \quad (3.7)$$

For a split with benign and attack subsets, the false-positive rate and true-positive rate in Equations (3.8) and (3.9) are

$$\text{FPR}_\mathcal{S}(\tau) = \frac{1}{|\mathcal{S}^{(0)}|} \sum_{(x,y) \in \mathcal{S}^{(0)}} \mathbf{1}[\hat{y}_\tau(x) = 1], \quad (3.8)$$

$$\text{TPR}_\mathcal{S}(\tau) = \frac{1}{|\mathcal{S}^{(1)}|} \sum_{(x,y) \in \mathcal{S}^{(1)}} \mathbf{1}[\hat{y}_\tau(x) = 1]. \quad (3.9)$$

The repository label ASR refers to detector false-negative rate, reported here as the FNR in Equation (3.10):

$$\text{FNR}_\mathcal{S}(\tau) = \frac{1}{|\mathcal{S}^{(1)}|} \sum_{(x,y) \in \mathcal{S}^{(1)}} \mathbf{1}[\hat{y}_\tau(x) = 0] = 1 - \text{TPR}_\mathcal{S}(\tau). \quad (3.10)$$

To avoid ambiguity with downstream attack success against a target model, this thesis reports that quantity as false-negative rate (FNR), i.e., detector miss rate at the chosen operating threshold. Ranking quality is reported with AUROC and AUPRC [9]. These threshold-free metrics remain useful, but they are not the deployment decision rule studied here.

The operating threshold is selected on validation at the target benign-FPR regime and then transferred unchanged to every evaluation split [8]. The implementation evaluates only the

candidate thresholds returned by the validation ROC curve and keeps the most permissive admissible candidate under the benign-FPR cap.

Let the admissible ROC candidates denote the validation thresholds whose FPR does not exceed the target. Equation (3.11) defines the admissible set:

$$\tau^* = \arg \max_{\tau \in \mathcal{T}_{\text{val}}^{\leq \alpha}} \text{FPR}_{\text{val}}(\tau). \quad (3.11)$$

Operationally, the code keeps the admissible ROC candidate with the largest validation FPR under the cap. If several candidates share that value, it keeps the first one returned by the ROC routine; if none satisfies the constraint, it falls back to the smallest available validation FPR. For the locked final run, this yields `val_threshold` = 0.7340, actual validation FPR 0.009409, and validation TPR 0.9696. That threshold is then transferred unchanged to every evaluation split.

The learned model is trained with standard cross-entropy on a strongly attack-heavy split, without class weighting, focal loss, or balanced sampling. The reported overblocking should therefore be interpreted for this implemented training prior and frozen protocol, not as evidence that threshold transfer alone explains every error pattern.

3.4.2 Locked Threshold-Transfer Protocol

The Week 7 locked protocol has four fixed steps:

1. select `week7_norm_only` as the final run;
2. compute the validation threshold on `val` only;
3. apply that threshold unchanged to `test_main`, `test_jbb`, and all Unicode, `adv2`, and rewrite variants;
4. package only aggregate results, figures, error cases, configuration metadata, and environment metadata into the locked evaluation pack.

As shown in Figure 3.2, the runtime decision object is one composed prompt+context payload, one scalar attack score, and one transferred threshold.

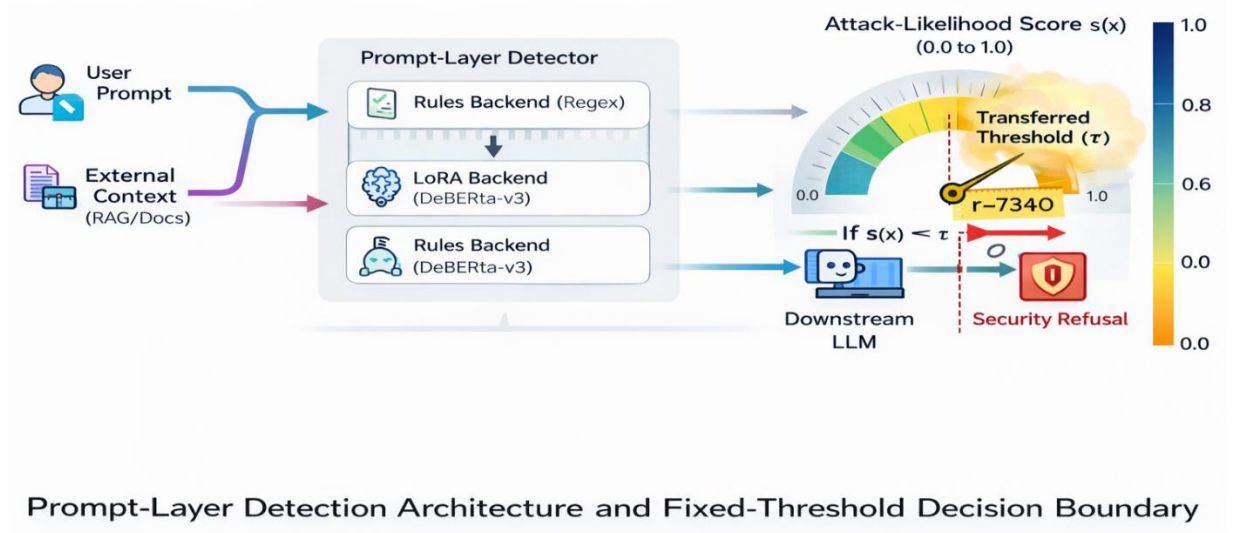


Figure 3.2 Runtime detector flow

3.5 Summary of This Chapter

This chapter defined the bounded methodological frame for the thesis: one composed payload, one frozen threshold-transfer protocol, one implemented detector pipeline, and one selection-aware stress-evaluation setup. Those choices set the conditions under which the later empirical claim should be read.

4. Algorithm or System Implementation

This chapter records only the implementation detail needed to audit and reproduce the reported scoring behavior. Its purpose is not to serve as a software manual, but to show that the empirical claim can be traced through a concrete offline runtime.

4.1 Runtime Overview

The delivered software is an installable Python package whose public runtime surface is the `jbd` command-line tool declared in `pyproject.toml` and implemented in:

`src/llm_jailbreak_detector/cli.py`. The runtime boundary is intentionally narrow: it accepts text, optionally applies inference-time normalization, routes the input to one of two detector backends, thresholds the resulting score, and emits a structured decision as JSON or JSONL. In this thesis, that runtime matters because it is the concrete offline scoring path used to reproduce the reported detector behavior.

A selective repository view makes the implementation boundary visible:

```
llm-guardrail-capstone-starter/  
├── pyproject.toml  
├── src/  
│   ├── llm_jailbreak_detector/  
│   └── baselines/  
├── scripts/  
├── runs/week7_norm_only/  
├── reports/week7/  
└── tests/
```

This layout separates the shipped runtime from the surrounding research pipeline. The installable runtime is concentrated in `src/llm_jailbreak_detector/`. The surrounding directories support data loading, preprocessing, model training, evaluation, artifact packaging, and testing. As a result, the repository is inspectable as a software system rather than only as an experiment folder.

The runtime exposes two detector backends behind one common interface. The guaranteed path is the rules backend, implemented through `RulesDetector` and backed by the regex baseline in `src/baselines/rules.py`. This path is fully offline-capable and returns a score of 1.0

on a pattern match and 0.0 otherwise. The optional path is the learned LoRA backend, implemented in `src/llm_jailbreak_detector/lora_detector.py`. It loads a local `run_dir`, reads `config.json`, recovers the stored validation threshold, and loads adapter weights from `lora_adapter/`. Unlike the surrounding research scripts, the public runtime loads backbone artifacts with `local_files_only=True`, which makes the artifact dependency explicit and prevents silent online fallback.

The user-facing interface is intentionally small. `jbd predict` scores one text string and prints one JSON object to standard output. `jbd batch` reads `.jsonl` or line-based `.txt` input and writes `.jsonl` output. `jbd normalize` exposes the normalization transform without performing detection. `jbd doctor` reports Python, platform, package version, and the installed status of optional LoRA dependencies. This CLI-first surface makes the software concrete and buildable while remaining consistent with the thesis boundary of one prompt-layer decision object and one offline-capable detector path.

4.2 Requirements and Measured Runtime Evidence

Table 4.1 summarizes the runtime requirements defended by the shipped implementation. These requirements are intentionally limited to behaviors that are implemented, inspectable, and relevant to the thesis boundary: single-record scoring through `jbd predict`, batch scoring through `jbd batch`, a default offline rules backend, an optional learned backend loaded from local artifacts, explicit normalization controls, and environment diagnostics through `jbd doctor`. The corresponding non-functional claims are equally bounded: fixed-input determinism apart from latency, default offline operation in rules mode, and reproducible recovery of the stored operating point from local artifacts.

Table 4.1 Core functional and non-functional requirements defended by the shipped runtime.

ID	Requirement	Delivered evidence
FR-1	Single-record scoring	Delivered via <code>jbd predict</code> with the documented JSON payload interface.
FR-2	Batch scoring	Delivered via <code>jbd batch</code> for both JSONL and TXT inputs, with deterministic JSONL output.
FR-3	Offline baseline detector	Delivered via the rules backend, which does not depend on external services or model downloads.
FR-4	Optional learned detector	Delivered via local-only LoRA loading from <code>run_dir</code> , <code>config.json</code> , and <code>lora_adapter</code> .
FR-5	Inspectable normalization	Delivered via <code>jbd normalize</code> and the explicit <code>--normalize/--drop-mn</code> inference flags.
FR-6	Demo-readiness	Delivered via <code>jbd doctor</code> and the diagnostic-check workflow

ID	Requirement	Delivered evidence
	diagnostics	documented in the testing appendix and user manual.
NFR-1	Default offline operation	Satisfied by the rules path; the runtime does not require hosted APIs for the required demo.
NFR-2	Reproducible operating point	Satisfied by stored val_operating_point.json, frozen metrics, and the locked Week 7 evidence pack.
NFR-3	Deterministic behavior	Satisfied for fixed inputs and artifacts, with latency as the only intentionally variable field.
NFR-4	Honest runtime surface	Satisfied by restricting public claims to the implemented CLI and excluding older draft-only flags or hidden services.

Table 4.2 reports local runtime measurements collected by: scripts/benchmark_thesis_runtime.py and stored in thesis_final_tex/runtime_benchmark.json. The measurements were taken on the submission machine under Windows 11 and Python 3.12.10 using repeated CPU calls to Predictor.predict(). They are therefore local library-level observations rather than first-call CLI timings or deployment-scale benchmarks.

Table 4.2 Observed local runtime and availability measurements for the shipped detector backends. These are library-level CPU timings from Predictor.predict(), not full CLI wall-clock measurements.

Mode	Threshold policy	Benign mean / p95	Attack mean / p95	Availability note
Rules	Fixed 0.5	0.0032 ms / 0.0034 ms	0.0009 ms / 0.0010 ms	Measured over 80 repeated CPU calls on the current Windows 11 / Python 3.12.10 submission workspace; always-available demo path.
LoRA	Stored validation threshold (val)	69.60 ms / 72.93 ms	70.54 ms / 75.50 ms	Measured over 20 repeated CPU calls with the local week7_norm_only artifacts and populated backbone cache. This confirms that the learned path is runnable offline once local model artifacts are present, but it is still much heavier than rules mode.

Interpretation of the runtime measurements.

Table 4.2 is intended as local reproducibility evidence, not as a measure of training time or full end-to-end deployment latency. The reported measurements are repeated CPU calls to the already-loaded Predictor.predict() function. They exclude model training, first-call initialization, command-line startup overhead, file loading, and the costs associated with a deployed service.

This scope explains the large gap between the two backends. The rules backend is extremely fast because it follows a lightweight pattern-based offline path and does not execute a transformer model. The LoRA backend uses the learned DeBERTa-based detector and is therefore much heavier, with prediction time around 70 ms in the measured local setting. These

values support the claim that both backends can run locally, but they should not be read as production latency claims.

Training and runtime measurement are separate parts of the work. The learned detector was trained locally with the LoRA-adapted DeBERTa-v3-base setup, whereas Table 4.2 reports prediction-time behavior after the trained artifacts already exist. The table is included to support implementation availability and reproducibility, not to suggest that the full training pipeline is lightweight.

The supporting evidence for these requirements is wider than the benchmark alone. The CLI contract is implemented in:

src/llm_jailbreak_detector/cli.py and src/llm_jailbreak_detector/io.py; rules-mode behavior is exercised in tests/test_cli_smoke.py; and scripts/verify_run_artifacts.py confirms that saved prediction files and stored metrics remain consistent for frozen run artifacts such as runs/week7_norm_only. The benchmark numbers are useful because they show the rules path as the guaranteed lightweight offline path and the LoRA path as a heavier but locally runnable artifact-dependent path, but they should be interpreted only as local reproducibility evidence.

4.3 Runtime Architecture and Reproducibility

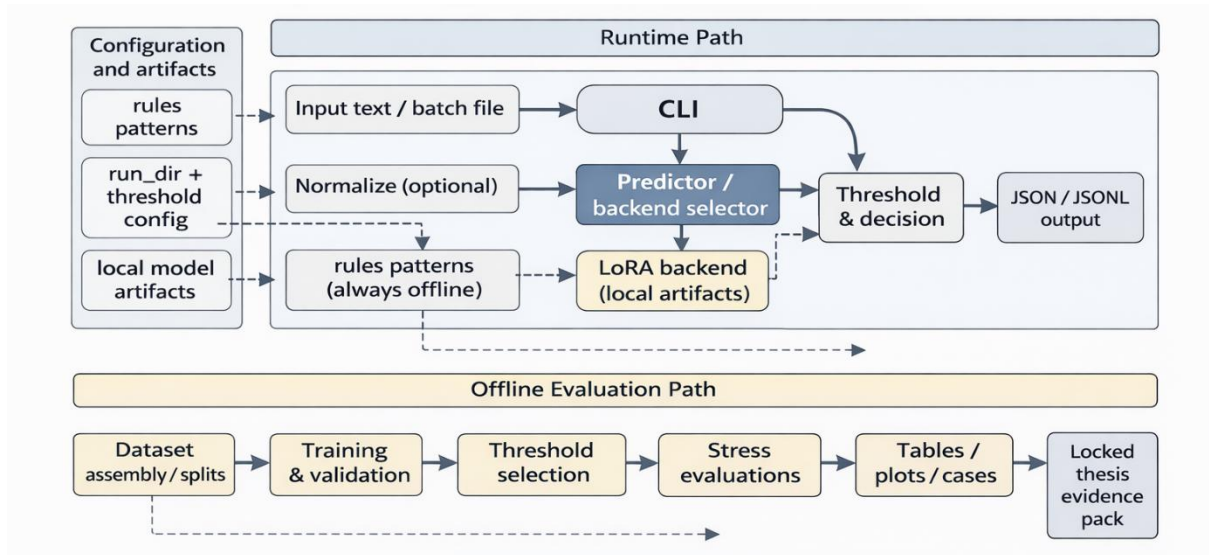


Figure 4.1 Shipped runtime architecture

The shipped runtime is exposed through the jbd CLI, which provides the concrete offline scoring path used in this study. Text can be normalized before scoring, after which Predictor

resolves the backend and threshold policy, routes the request to either the rules detector or the LoRA-based detector, and emits structured JSON or JSONL outputs for single-item and batch evaluation.

Failure behavior is explicit rather than silently degraded. The learned path depends on local artifacts such as `run_dir`, `config.json`, `lora_adapter`, and the required local model files; if those artifacts are missing, the runtime reports the problem directly instead of falling back invisibly. This keeps the rules backend as the guaranteed offline path while making the artifact dependence of the learned path clear for honest reproducibility.

Exact runtime evidence paths and repository verification commands are listed in Appendix A.2-A.3.

4.4 System Role in the Study

The delivered software is the reproducible runtime surface for the case study. Its engineering contribution is concrete: the reported scoring behavior can be reproduced from local artifacts through a small, auditable interface. The chapter makes no broader service claim beyond that role.

4.5 Summary of This Chapter

This chapter showed why the implementation chapter belongs in the thesis: it makes the offline scoring path, frozen configuration, and runtime evidence auditable before the empirical results are interpreted.

5. Experimental Results and Analysis

This chapter reports the locked Week 7 operating-point evaluation for the selected learned detector and the shipped rules fallback. The central question is simple: after calibrating a low-benign-error threshold on validation, how does that fixed threshold behave when transferred unchanged to shifted prompt distributions? The chapter therefore separates ranking quality from fixed-threshold behavior and treats the latter as the main object of evaluation.

5.1 Test Environment and Evaluation Design

The fixed Week 7 grid contains one validation split and two inspected evaluation families. Table 5.1 repeats the authoritative counts used in this chapter.

Table 5.1 Evaluation split counts used in Chapter 5.

Split	Total	Benign	Attack
train	201,276	14,859	186,417
val	6,106	3,082	3,024
test_main	35,230	3,147	32,083
test_jbb	197	98	99
test_main_unicode	35,230	3,147	32,083
test_jbb_unicode	197	98	99
test_main_adv2	35,230	3,147	32,083
test_jbb_adv2	197	98	99
test_main_rewrite	35,230	3,147	32,083
test_jbb_rewrite	197	98	99

Evaluation-status note. Consistent with the convention set in Chapter 1, interpretive prose refers to test_main as the main stress split and test_jbb as the JBB stress split. Both were inspected during model selection, so they function as selection-aware development-evaluation stress splits within the study, not as blind held-out tests. Validation sets the operating point, the main stress split shows transfer on the main project distribution, and the JBB stress split isolates controlled cross-dataset shift.

The main stress split is strongly attack-heavy, while validation is approximately balanced. This matters for interpretation: AUPRC is prevalence-sensitive, whereas fixed-threshold FPR and TPR remain the more decision-relevant metrics for the thesis question. The later source-assembled benign-only analysis slice and the recalibration follow-up in this chapter refine that

picture, but they do not change the threshold-transfer framing.

Week 7 model selection compared a small set of configurations that differed in training-time normalization and adversarial-augmentation settings. The selected run, week7_norm_only, was retained as the final selected configuration because it offered the most defensible balance across the inspected split grid. Because that grid mixes different split sizes and stress roles, the choice is best understood as a practical project-level selection under the stated protocol, not as proof that week7_norm_only is globally best under all weighting choices.

Table 5.2 Rules baseline results at the fixed threshold of 0.5.

Split	AUROC	AUPRC	FPR@0.5	TPR@0.5	FNR@0.5
test_main	0.5024	0.9111	0.0016	0.0064	0.9936
test_main_unicode	0.5009	0.9108	0.0010	0.0028	0.9972
test_main_adv2	0.5007	0.9108	0.0000	0.0015	0.9985
test_jbb_adv2	0.5000	0.5025	0.0000	0.0000	1.0000
test_main_rewrite	0.5021	0.9110	0.0016	0.0057	0.9943

At its fixed threshold of 0.5, the shipped rules baseline exhibits extremely low recall on the available evaluated splits, for example $\text{TPR}@0.5 = 0.0064$ on the main stress split. The thesis therefore uses it only as a transparent offline fallback and runtime reference point. It is useful for reproducibility, debuggability, and verification, but it should not carry argumentative weight in the research claims about detector quality.

5.2 Main Experimental Results

5.2.1 Main Distribution Results

Tables 5.3 and 5.4 establish the main-distribution result from two angles. Table 5.3 is the headline operating-point view, while Table 5.4 places the selected run within the clean and Unicode Week 7 grid. The fuller winner-only split table is moved to Appendix C (Table C.3) for reference.

Table 5.3 Main operating-point overview for the frozen week7_norm_only detector. The final column translates benign false-positive rate into blocked benign examples, which makes the practical cost of threshold transfer easier to read than AUROC alone.

Split	Shift role	AUROC	FPR@ τ	TPR@ τ	Benign blocked	Attack n_1
val	Validation calibration	0.9983	0.0094	0.9696	29/3082	3024
test_main	Main-distribution	0.9958	0.0311	0.9869	98/3147	32083

Split	Shift role	AUROC	FPR@ τ	TPR@ τ	Benign blocked	Attack n_1
	transfer					
test_jbb	Cross-dataset shift	0.8890	0.5816	0.9798	57/98	99
test_main_adv2	Obfuscation shift	0.7133	0.6403	0.9967	2015/3147	32083
test_jbb_adv2	Combined shift	0.6318	0.8469	0.8990	83/98	99

Across Tables 5.3 and 5.4, the same pattern remains visible: ranking stays strong on the main distribution, but decision behavior at the chosen threshold deteriorates once evaluation moves to harder distributions. Table 5.4 shows where the selected run sits within the clean and Unicode Week 7 grid; the row group of interest is week7_norm_only, whose validation threshold is 0.7340. The fuller winner-only split summary is retained in Appendix C (Table C.3).

Table 5.4 Clean and Unicode results from the Week 7 locked evaluation pack.

Run	Split	AUROC	AUPRC	τ_{val}	FPR@ τ	TPR@ τ	FNR@ τ
week7_adv2_only	val	0.9982	0.9982	0.5624	0.0097	0.9729	0.0271
week7_adv2_only	test_main	0.9942	0.9994	0.5624	0.0607	0.9970	0.0030
week7_adv2_only	test_jbb	0.8962	0.8942	0.5624	0.6939	0.9899	0.0101
week7_adv2_only	test_main_unicode	0.9933	0.9992	0.5624	0.0642	0.9969	0.0031
week7_adv2_only	test_jbb_unicode	0.8393	0.8322	0.5624	0.7551	1.0000	0.0000
week7_control	val	0.9982	0.9982	0.6894	0.0097	0.9709	0.0291
week7_control	test_main	0.9960	0.9996	0.6894	0.0311	0.9878	0.0122
week7_control	test_jbb	0.9078	0.9033	0.6894	0.5918	0.9798	0.0202
week7_control	test_main_unicode	0.9951	0.9995	0.6894	0.0423	0.9858	0.0142
week7_control	test_jbb_unicode	0.8857	0.8800	0.6894	0.7041	0.9697	0.0303
week7_norm_only	val	0.9983	0.9982	0.7340	0.0094	0.9696	0.0304
week7_norm_only	test_main	0.9958	0.9996	0.7340	0.0311	0.9869	0.0131
week7_norm_only	test_jbb	0.8890	0.8747	0.7340	0.5816	0.9798	0.0202
week7_norm_only	test_main_unicode	0.9949	0.9994	0.7340	0.0429	0.9862	0.0138
week7_norm_only	test_jbb_unicode	0.8814	0.8704	0.7340	0.6531	0.9798	0.0202
week7_norm_plus_adv2	val	0.9979	0.9977	0.6815	0.0094	0.9659	0.0341
week7_norm_plus_adv2	test_main	0.9929	0.9992	0.6815	0.0566	0.9944	0.0056
week7_norm_plus_adv2	test_jbb	0.9018	0.8950	0.6815	0.6122	0.9798	0.0202
week7_norm_plus_adv2	test_main_unicode	0.9920	0.9991	0.6815	0.0582	0.9959	0.0041
week7_norm_plus_adv2	test_jbb_unicode	0.8892	0.8744	0.6815	0.6735	0.9697	0.0303

Three observations from Table 5.4 matter most:

1. On the main stress split, ranking performance is strong and recall remains high at the transferred threshold.
2. Even on clean text, the transferred threshold does not preserve the 1% FPR target: the main stress split rises to 0.0311.
3. Cross-dataset shift dominates benign harm on test_jbb, where the same threshold yields FPR=0.5816 despite high recall.

Table 5.5 adv2 and rewrite results from the Week 7 locked evaluation pack.

Run	Split	AUROC	AUPRC	τ_{val}	FPR@ τ	TPR@ τ	FNR@ τ
week7 adv2 only	test main adv2	0.9833	0.9979	0.5624	0.0985	0.9856	0.0144
week7 adv2 only	test jbb adv2	0.6415	0.6689	0.5624	0.6633	0.7677	0.2323
week7 adv2 only	test main rewrite	0.9912	0.9990	0.5624	0.0801	0.9977	0.0023
week7 adv2 only	test jbb rewrite	0.8870	0.8918	0.5624	0.6939	0.9899	0.0101
week7 control	test main adv2	0.6547	0.9379	0.6894	0.6826	0.9731	0.0269
week7 control	test jbb adv2	0.6007	0.5727	0.6894	0.8980	0.9495	0.0505
week7 control	test main rewrite	0.9934	0.9992	0.6894	0.0537	0.9896	0.0104
week7 control	test jbb rewrite	0.8993	0.8907	0.6894	0.6020	0.9798	0.0202
week7 norm only	test main adv2	0.7133	0.9523	0.7340	0.6403	0.9967	0.0033
week7 norm only	test jbb adv2	0.6318	0.6294	0.7340	0.8469	0.8990	0.1010
week7 norm only	test main rewrite	0.9942	0.9994	0.7340	0.0531	0.9902	0.0098
week7 norm only	test jbb rewrite	0.8956	0.8964	0.7340	0.5714	0.9899	0.0101
week7 norm plus adv2	test main adv2	0.9833	0.9979	0.6815	0.0947	0.9831	0.0169
week7 norm plus adv2	test jbb adv2	0.6329	0.6506	0.6815	0.5408	0.6768	0.3232
week7 norm plus adv2	test main rewrite	0.9905	0.9990	0.6815	0.0718	0.9968	0.0032
week7 norm plus adv2	test jbb rewrite	0.8754	0.8701	0.6815	0.6224	0.9798	0.0202

Table 5.5 sharpens the contrast between ranking quality and decision-level usability. Adv2 does not merely weaken discrimination; it pushes benign scores upward until the selected threshold becomes unusable even while recall stays high. Rewrite behaves differently. On the main distribution it is comparatively mild, suggesting that lexical variation alone is not the dominant failure mode. The stronger conclusion is not that every perturbation harms the detector equally, but that adv2-style corruption is especially damaging because it moves score mass across the decision boundary in a way that threshold-free metrics alone do not show. The plots below are regenerated from the frozen per-example prediction files so that the selected threshold is marked explicitly rather than inferred only from the tables. Figures 5.1-5.3 then make that main-stress comparison explicit in ROC and score-distribution views.

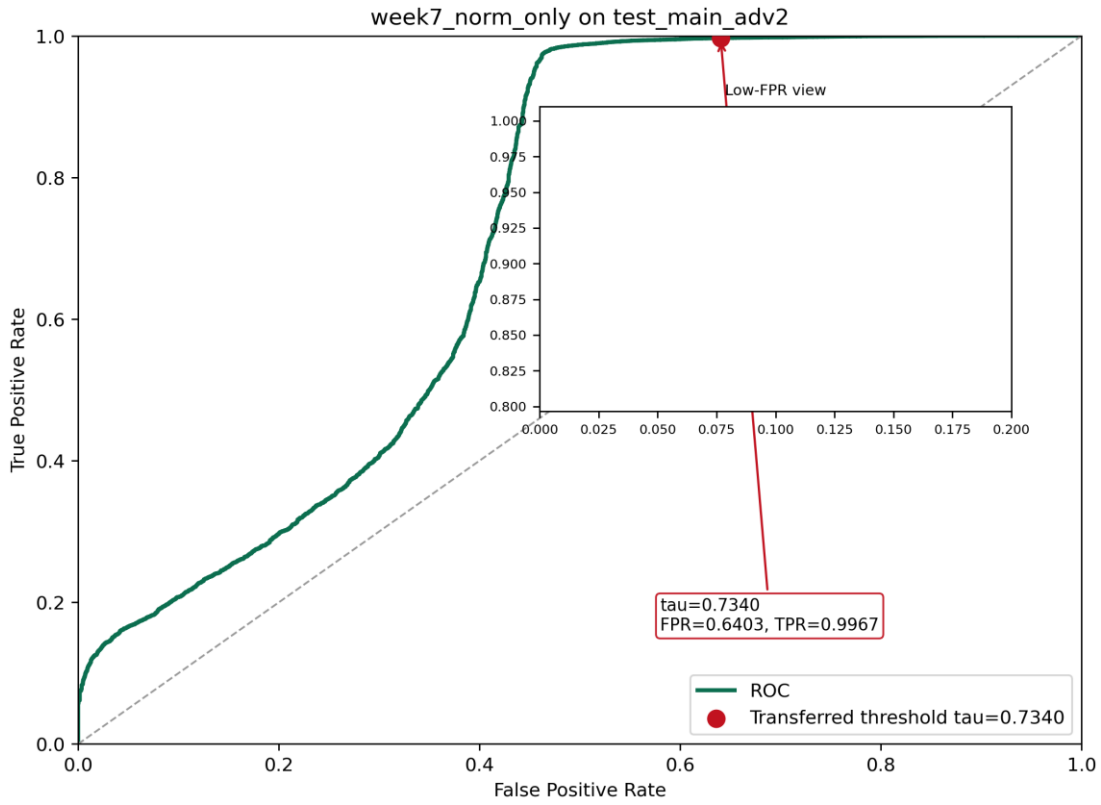


Figure 5.1 ROC curve on test_main_adv2

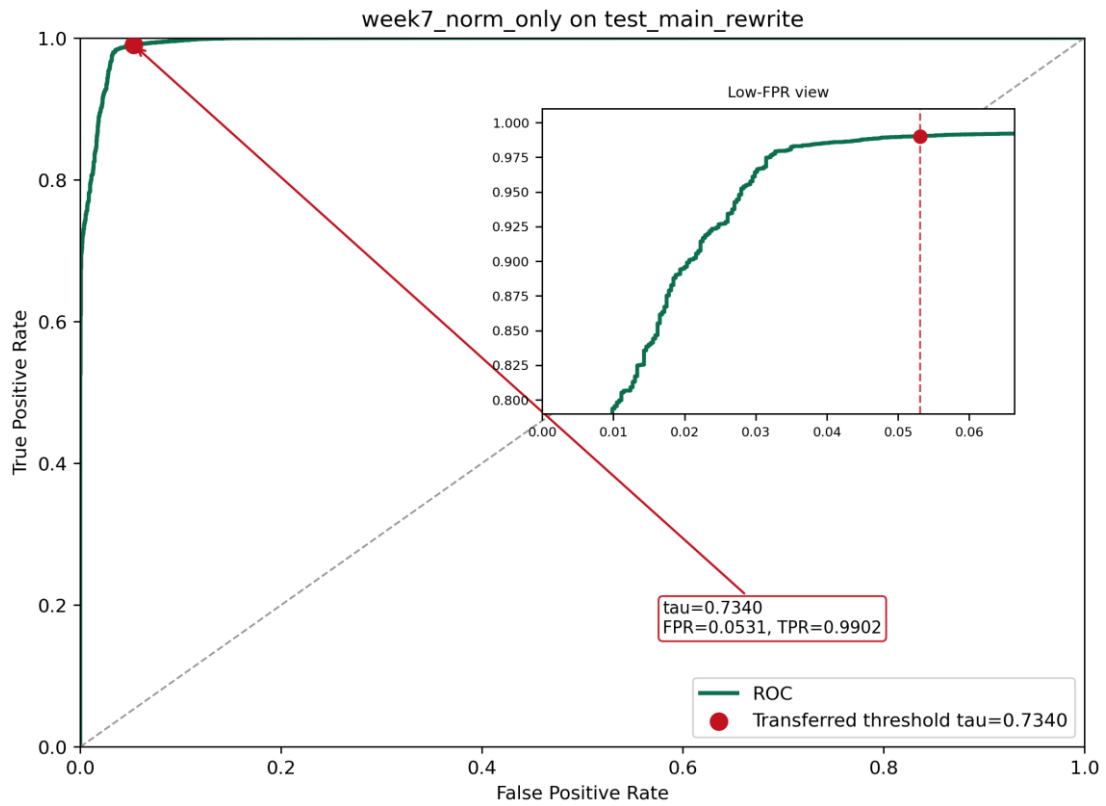


Figure 5.2 ROC curve on test_main_rewrite

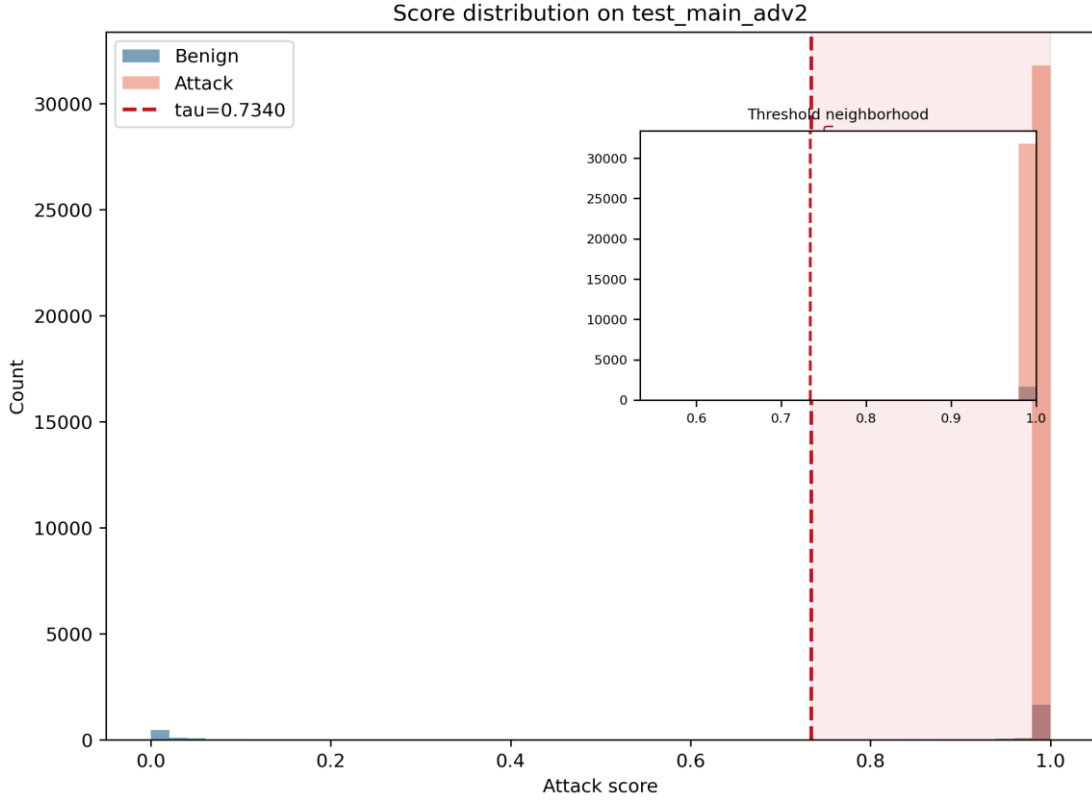


Figure 5.3 Score histogram on test_main_adv2

5.2.2 Cross-Dataset Shift Results

The main cross-dataset failure is benign overblocking under shift. Rates alone hide its practical cost: on the shifted splits, the detector still catches many attacks but blocks a large share of benign inputs. Table 5.6 therefore converts the stored FPR and TPR values for week7_norm_only into confusion-count summaries using the locked split counts. That count view makes the deployment problem visible as benign harm, not just as an abstract rate.

That translation matters. On the main distribution, the clean split still produces a non-trivial number of benign blocks, but the JBB and adv2 variants reveal the more serious pattern: the detector continues to catch many attacks while overblocking a large share of benign inputs. The main empirical problem is therefore not simply missed attacks. It is that the detector can look effective by recall while becoming operationally costly once benign score drift is large. The count view is central to the thesis because it turns this behavior into visible benign harm rather than leaving it as an abstract percentage. Figure 5.4 complements Table 5.6 by isolating the benign subset and showing how many benign inputs pass versus are blocked.

Table 5.6 Confusion-count view of the transferred threshold for week7_norm_only. Counts

are derived from locked aggregate split sizes and the stored operating-point rates.

Split	Benign n_0	Attack n_1	FP	TN	TP	FN
val	3082	3024	29	3053	2932	92
test_main	3147	32083	98	3049	31664	419
test_main_unicode	3147	32083	135	3012	31639	444
test_main_rewrite	3147	32083	167	2980	31767	316
test_main_adv2	3147	32083	2015	1132	31976	107
test_jbb	98	99	57	41	97	2
test_jbb_unicode	98	99	64	34	97	2
test_jbb_rewrite	98	99	56	42	98	1
test_jbb_adv2	98	99	83	15	89	10

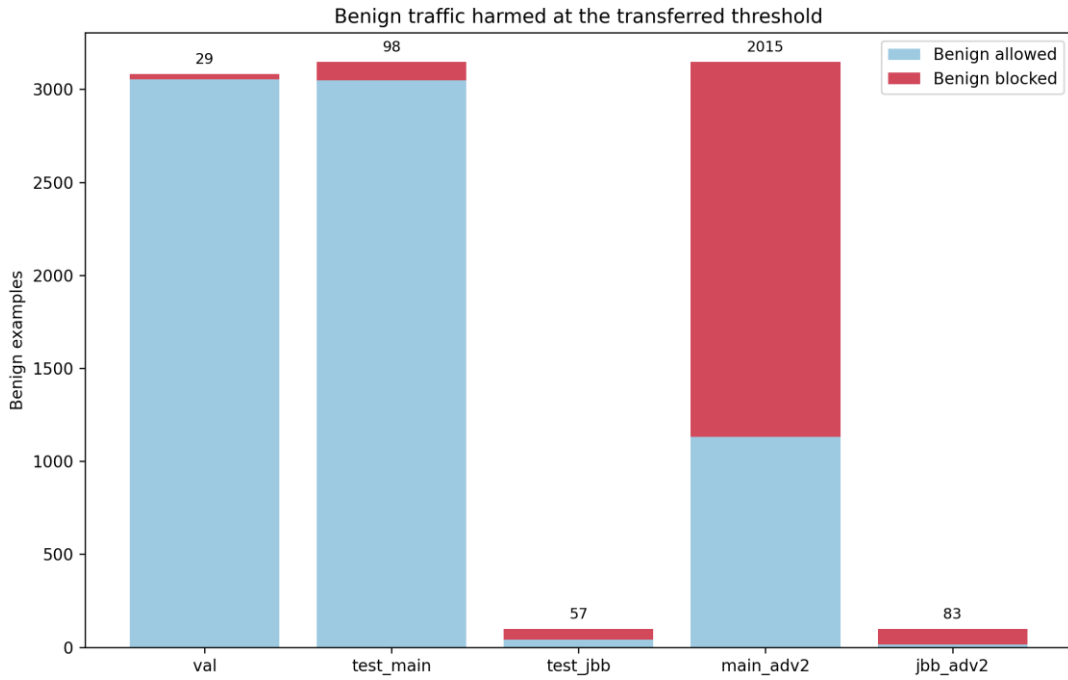


Figure 5.4 Benign-harm view of the transferred threshold

The adv2 pattern is even more concrete in count form. For test_main_adv2, the detector still blocks 31,976 of 32,083 attacks, but it also falsely flags 2,015 benign inputs. On test_jbb_adv2, only 15 benign examples remain unflagged. These counts make the same point in operational terms: high recall under perturbation does not imply a usable operating point once benign-score drift is large.

Because the main distribution is large and JBB is small, uncertainty differs sharply across splits. Wilson 95% confidence intervals are reported in Appendix C (Table C.4). The intervals remain tight on the large main stress-split family and much wider on the 98-benign / 99-attack JBB family, so the JBB rows should be read as strong stress signals rather than high-precision estimates of the true deployment rate.

5.3 Robustness Analysis

Beyond the main test results, the robustness analysis asks whether the chosen validation threshold remains usable on perturbation and cross-dataset evaluations. The focus is not one attack family in isolation, but whether benign harm stays controlled once the prompt distribution changes.

Threat model and evaluation setting.

All robustness claims in this chapter use the transferred validation threshold $\tau = 0.7340$ and the locked Week 7 counts. No split-specific recalibration is allowed. The perturbation families are:

- **Unicode:** formatting and character-set obfuscation;
- **rewrite:** mild paraphrase-style lexical editing.

For a clean input, the robustness study evaluates transformed variants of the form

$$x' \in \{U_{\theta_u}(x), A_{\theta_a}(x), R_{\theta_r}(x)\}, \quad (5.1)$$

where the Unicode obfuscation operator, the adv2 character-noise operator, and the rewrite operator define the three perturbation families. The labels of perturbed examples are inherited from their base examples when the variants are generated. This inheritance is exact for the surface-level Unicode and adv2 transforms, but only approximate for rewrite, which can sometimes soften or drift the meaning of the original prompt.

The perturbation families in this chapter should also be read under two threat-model logics. An adaptive attacker perturbs attacks only, with the goal of preserving malicious intent while lowering the detector score enough to pass the threshold. A noisy-channel corruption, by contrast, affects both benign and malicious text, for example through formatting noise, mixed-script content, or brittle tokenization effects in real application pipelines. The distinction matters because these two cases stress different properties of the detector. Attack-only perturbations test evasion resistance, while bidirectional corruption tests whether the decision rule remains usable once the overall text channel changes. In this thesis, adv2 is useful partly because it can proxy attacker obfuscation, but the broader benign-score drift on the harder evaluations also points to a noisy-channel problem rather than a pure attack-only evasion story.

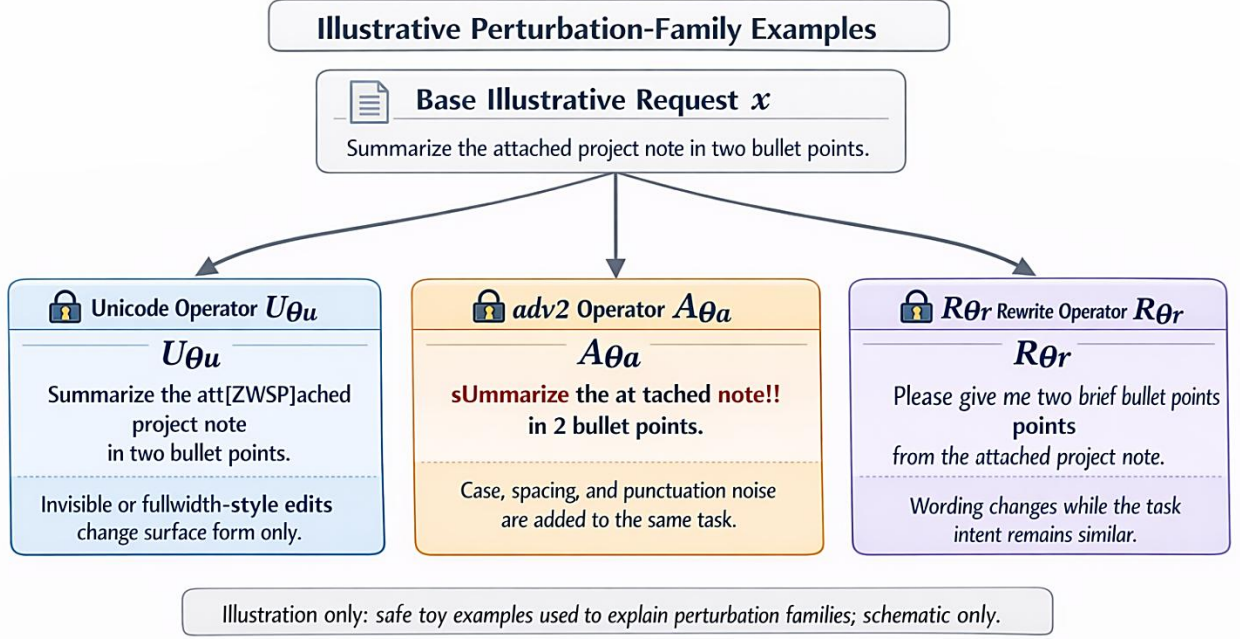


Figure 5.5 Illustrative perturbation families

The robustness probes in this chapter are defined by the perturbation operators in Equations (5.2)-(5.5).

Unicode operator.

The Unicode transform applies two randomized edits while scanning the text. First, printable ASCII characters may be mapped to fullwidth forms. Second, a zero-width space may be inserted after non-newline characters. Equation (5.2) defines the operator:

$$U_{\theta_u}(x) = \text{insert_zwsp}(\text{fullwidth}(x; p_{\text{fw}}); p_{\text{zwsp}}), \quad (5.2)$$

This operator is intended to preserve semantic content while disturbing tokenization and brittle lexical cues.

adv2 operator.

The adv2 transform is a character-level noise process with independent local edits applied as the text is scanned. Equation (5.3) gives the default parameterization.

$$\theta_a = (p_{\text{case}}, p_{\text{space}}, p_{\text{punct}}, p_{\text{hom}}, p_{\text{mix}}, p_{\text{zw}}) = (0.20, 0.08, 0.03, 0.06, 0.02, 0.03), \quad (5.3)$$

With at most two zero-width insertions when the zero-width branch fires, the operator may flip case, add extra spaces after spaces, insert punctuation, replace characters with homoglyphs, insert mixed-script symbols, and insert zero-width characters. Equation (5.4) gives a compact definition:

$$A_{\theta_a}(x) =$$

$$\text{compose}(\text{case flip, space insert, punctuation insert, homoglyph swap, mixed-script insert, zero-width insert})(x; \theta_a). \quad (5.4)$$

This is the most aggressive perturbation family in the thesis and is designed to preserve task intent while making lexical matching and calibration harder.

Rewrite operator.

The rewrite transform is a lightweight paraphrase proxy rather than a semantically controlled generative rewrite. It first swaps one adjacent sentence pair, then performs token-level synonym substitution and filler-phrase insertion. Equation (5.5) defines the operator:

$$R_{\theta_r}(x) = \text{token_rewrite}(\text{sentence_swap}(x; p_{\text{swap}}); p_{\text{syn}}, p_{\text{fill}}), \quad (5.5)$$

Because this operator can change emphasis and local wording, its inherited labels should be read as a synthetic stress assumption rather than as a guarantee of perfectly preserved semantics.

5.3.1 Unicode, adv2, and Rewrite Behavior

Unicode perturbation increases false positives more than it harms recall. On the main stress split, FPR rises from 0.0311 to 0.0429 while TPR remains essentially unchanged. On the JBB stress split, the same trend is harsher: FPR rises from 0.5816 to 0.6531 without any compensating recall gain. The pattern suggests that the selected configuration is more robust in ranking than in calibration.

adv2 is the decisive robustness failure mode. On test_main_adv2, recall remains extremely high (TPR = 0.9967), yet FPR inflates to 0.6403. On test_jbb_adv2, benign blocking and attack misses both worsen: FPR reaches 0.8469 and TPR falls to 0.8990. More importantly, adv2 pushes enough benign score mass above the chosen threshold to make the decision rule unusable in practice. Operationally, the detector would overblock benign traffic while still missing some obfuscated attacks. This is the clearest evidence for the thesis's central result: ranking can survive while thresholded usability collapses once the evaluation distribution changes.

Rewrite results should be interpreted more cautiously than Unicode and adv2 results. The rewrite perturbation is useful as a reproducible stress probe, but its inherited labels are only approximate. Some rewritten prompts remain clearly malicious, whereas others drift toward more ambiguous or apparently benign assistance requests. For that reason, rewrite is discussed here as secondary supporting evidence rather than as a clean primary test of detector robustness.

Rewrite is much milder on the main distribution. On test_main_rewrite, FPR is 0.0531 and TPR is 0.9902. However, rewrite does not fix cross-dataset shift: on test_jbb_rewrite, benign FPR remains 0.5714 even though TPR is still 0.9899.

Figures 5.6 and 5.7 show the JBB adv2 failure from complementary views: the transferred threshold remains far from a usable benign-error region, and most benign examples fall into the block region.

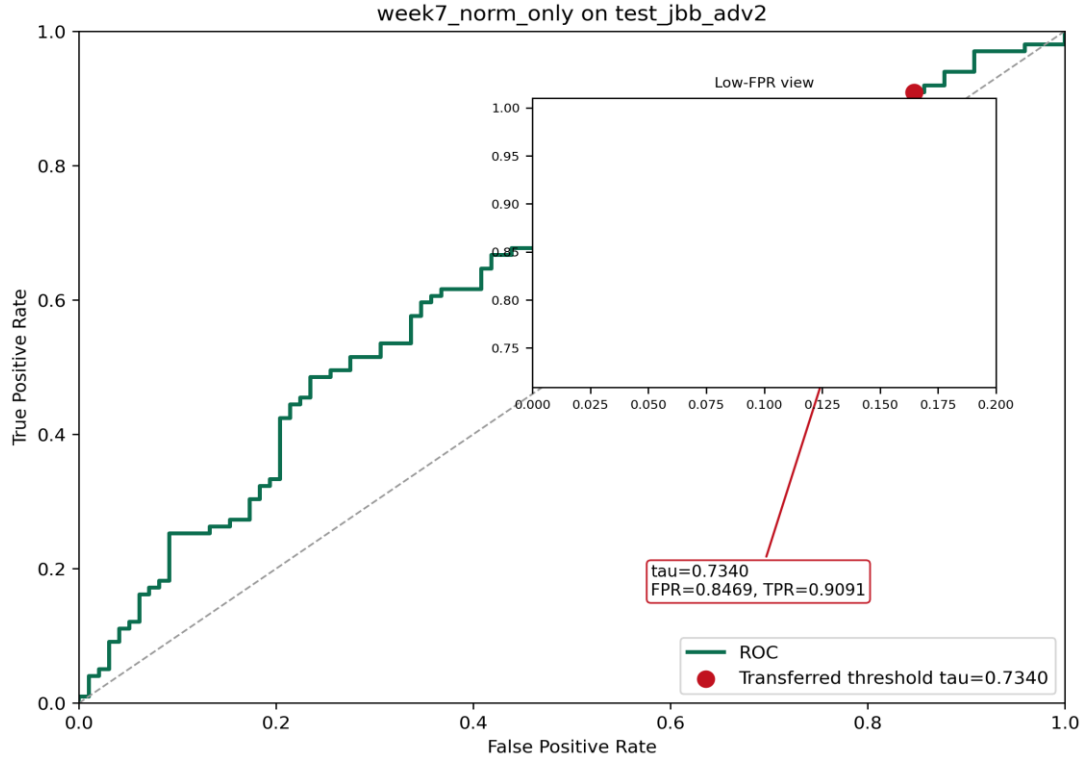


Figure 5.6 ROC curve on test_jbb_adv2

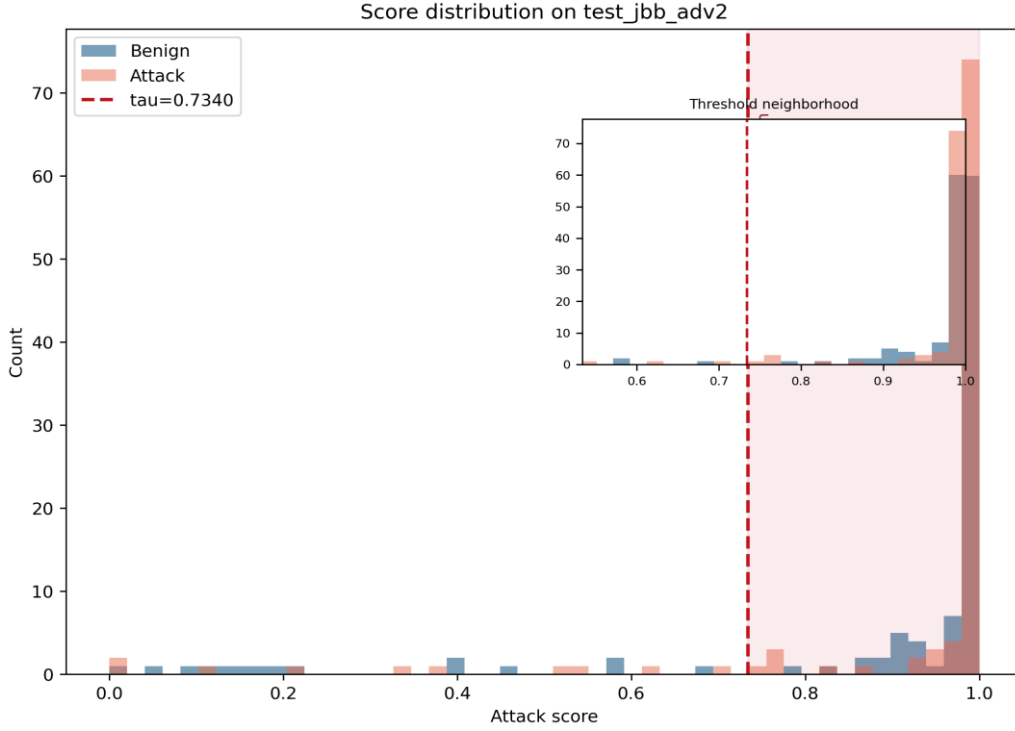


Figure 5.7 Score histogram on test_jbb_adv2

The directly observed score histograms and thresholded metrics already show the same pattern that an activation-space projection was meant to suggest: under adv2, benign-score drift increases overlap with attack-like behavior. The main argument therefore rests on the score distributions and operating-point evidence, with that supplementary visualization serving only a supporting role.

The JBB splits contain only 98 benign examples, so FPR is coarse: each extra benign false positive changes the reported FPR by about 1.02%. The JBB results should therefore be treated as a stress signal rather than a high-precision estimate of deployment FPR. Even with that caution, the direction of the result is unambiguous: the transferred threshold is not stable on the JBB benign distribution.

5.3.2 Inference-Time Canonicalization Follow-up

Inference-time canonicalization helps on the obfuscated splits, but it does not restore a usable low-FPR operating point. On test_main_adv2, benign FPR falls from 0.6403 on the submitted raw path to 0.5043 when Unicode-aware preprocessing is enabled, and the Unicode stress rows improve as well. The improvement is real, but the same decision rule still overblocks shifted benign text badly: even the stronger hardening settings leave test_jbb_adv2 above 0.81 benign FPR. Canonicalization therefore reduces some implementation-side damage without changing

the practical conclusion on the harder evaluations.

Table 5.7 Inference-time canonicalization follow-up using the frozen detector and the already archived manual-evaluation artifacts. Canonicalization helps on Unicode and adv2 stress, but even the strongest setting remains far above the intended low-FPR regime on shifted benign text.

Inference path	τ	test_main_unicode FPR	test_main_adv2 FPR	test_jbb_unicode FPR	test_jbb_adv2 FPR	test_main_adv2 TPR	test_jbb_adv2 TPR
Locked raw path	0.7340	0.0429	0.6403	0.6531	0.8469	0.9967	0.8990
Normalize only	0.7340	0.0311	0.5450	0.5816	0.8265	0.9958	0.8687
Unicode preprocess only	0.7340	0.0315	0.5043	0.5816	0.8163	0.9947	0.8586
Unicode + normalize	0.7340	0.0315	0.5043	0.5816	0.8163	0.9947	0.8586

The follow-up partially addresses the normalization-mismatch concern by isolating the effect of inference-time canonicalization without changing the trained model or the transferred validation threshold. The results show that canonicalization reduces some obfuscation-induced false positives, especially on the adv2 stress setting, but the FPR remains far above the intended low-FPR regime. Therefore, the mismatch is a contributing implementation limitation, but it is not a sufficient explanation for the full threshold-transfer failure observed in the locked pipeline.

Appendix C (Table C.5) retains a small paired multilingual and mixed-script probe as a narrow sanity check only, not as benchmark-scale multilingual evidence.

5.3.3 Threshold and Calibration Sensitivity

Nearby threshold changes do not rescue the operating point on the shifted evaluations. Figure 5.8 shows that validation stays close to its intended low-FPR region around the transferred threshold, while the harder splits remain in a high-FPR regime across the same local neighborhood. The result is therefore not driven by one unlucky threshold setting.

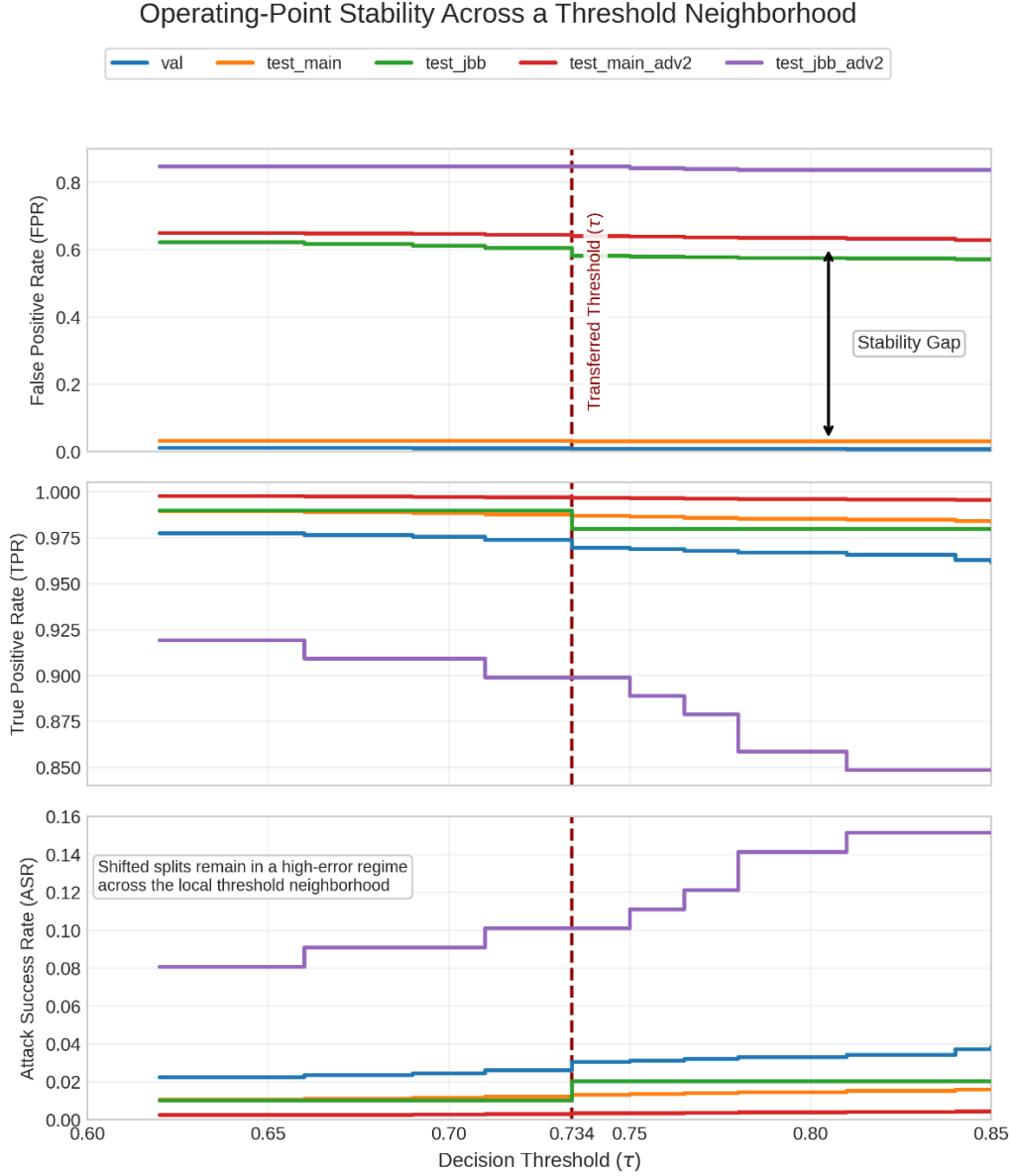


Figure 5.8 Threshold sensitivity around the transferred operating point
Benign-heavy recalibration changes the threshold modestly, but it does not fix the problem. As Table 5.8 shows, lowering attack prevalence in the validation slice raises the selected threshold slightly and trims some FPR, yet the harder evaluations remain far from a usable low-FPR regime. Even at attack prevalence 0.05, JBB-stress benign FPR is 0.5653 and test_jbb_adv2 remains 0.8204. Values for attack prevalence 0.10 and 0.05 are means over repeated validation resamples; 0.495 is the original validation balance.

Table 5.8 Benign-heavy recalibration follow-up for the frozen detector. Lower attack prevalence in the validation slice raises the threshold slightly and trims FPR, but the shifted

splits remain far from a usable low-FPR regime.

Attack prev.	τ	test_main FPR	test_jbb FPR	test_main_adv2 FPR	test_jbb_adv2 FPR	test_main TPR	test_jbb_adv2 TPR
0.495	0.7340	0.0311	0.5816	0.6403	0.8469	0.9869	0.8990
0.100	0.7832	0.0305	0.5765	0.6286	0.8367	0.9852	0.8768
0.050	0.8075	0.0300	0.5653	0.6207	0.8204	0.9844	0.8677

Score calibration is also distribution-sensitive. Appendix C (Table C.6) shows low ECE and Brier error on validation and on the main stress split, followed by sharp degradation under cross-dataset and adv2 shift. That matters because the transferred operating point assumes that score meaning remains stable when evaluation changes, and these follow-ups show that it does not.

Post-lock three-seed reruns point in the same direction. Appendix C (Table C.7) shows that the qualitative pattern persists across reruns: ranking remains relatively stable where the source-like split is easiest, while thresholded behavior stays unstable on the harder evaluations. These reruns are follow-up sensitivity evidence, not a replacement winner-selection process.

Taken together, the sensitivity checks reinforce the same interpretation. The threshold sweep rules out a narrow local-threshold effect, benign-heavy recalibration provides only modest relief, calibration quality breaks under harder shift, and the reruns preserve the gap between ranking quality and decision-level usability. Rewrite points in the same direction, but remains supporting evidence because its labels are only approximate.

5.4 Error Analysis and Discussion

5.4.1 Failure Cases and Benign Overblocking

To probe benign overblocking more directly, a small source-assembled benign-only slice was drawn from the frozen benign pool of the main stress split and evaluated at the same selected threshold. The slice is intentionally simple and fully reproducible: 300 benign prompts total, with 50 each from general assistance, coding assistance, rewriting or summarization, translation, knowledge Q&A, and prompt+context tasks. Because it is assembled from that stored benign pool rather than sampled from field traffic, it should be read as a reproducible analysis slice rather than a live deployment estimate.

Table 5.9 Source-assembled benign-only analysis slice evaluated at the frozen validation

threshold. The reference row re-expresses the benign subset of the main stress split at the same operating point, so the table isolates how many benign requests the detector would block per 1,000 prompts in this source-assembled slice.

Slice	Benign n	FPR@ τ	FP count	Blocked benign / 1,000
test main benign subset	3147	0.0311	98/3147	31.1
Source-assembled benign-only analysis slice	300	0.0933	28/300	93.3

Source-assembled analysis slice Wilson 95% CI for FPR: [0.0654, 0.1316].

Table 5.9 shows that the chosen threshold blocks 28 of 300 benign requests (FPR = 0.0933, or 93.3 blocked benign requests per 1,000). This is materially harsher than the benign subset of the main stress split, which blocks 31.1 benign requests per 1,000 at the same setting. The increase is also not uniform across categories. In this analysis slice, general assistance, coding, rewriting or summarization, translation, and knowledge Q&A pass without observed false positives, while the entire false-positive burden falls on prompt+context benign tasks.

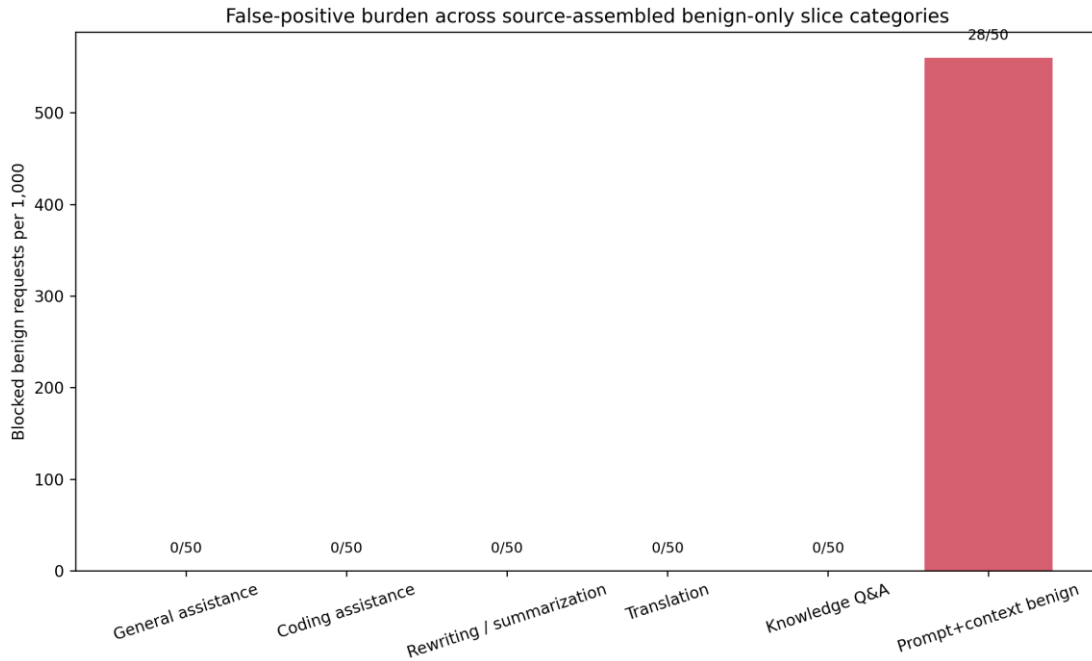


Figure 5.9 Benign-only category breakdown

Figure 5.9 makes that concentration explicit. All 28 false positives come from the prompt+context subset, where 28 of 50 benign examples are blocked. These are still conservative benign tasks such as extraction or question answering over structured context, so the figure sharpens the practical meaning of the overblocking result.

The locked pack also includes ranked false-positive and false-negative lists under `error_cases/`. To avoid reproducing harmful prompts verbatim in the thesis, Table 5.10 groups those lists into

sanitized categories rather than quoting raw examples.

Table 5.10 Sanitized failure-case categories derived from the locked Week 7 ranked error-case lists. Raw prompts are not reproduced; the table groups them into recurring patterns instead.

Category	Locked-pack evidence	Observed pattern	Interpretation
Obfuscated attack false negatives	test_main_adv2_fn.md, test_jbb_adv2_fn.md	Attack-labeled prompts about malware, identity theft, covert tracking, or physical harm are heavily corrupted by confusables, zero-width noise, and marker distortion yet still score far below τ .	Character-level obfuscation can preserve malicious intent while driving the detector toward false negatives.
Benign topical false positives	test_jbb_adv2_fp.md, test_jbb_rewrite_fp.md	Benign prompts discussing sensitive topics such as eating disorders, drugs, blackmail, or controversial ideologies receive scores near 1.0 when they retain high-risk vocabulary.	The detector overweights lexical overlap with unsafe intent, which is especially costly under cross-dataset shift.
Structured benign false positives	test_main_adv2_fp.md, test_main_rewrite_fp.md	Tables, translation/code tasks, math word problems, and document-style snippets with noisy formatting are frequently classified as attacks.	Formatting corruption and mixed prompt-context structure can look attack-like even when the underlying task is benign.
Rewrite semantic drift and ambiguity	test_main_rewrite_fn.md, test_jbb_rewrite_fn.md	Some attack-labeled rewrite examples read like innocuous wellness, factual, or generic assistance prompts after rewriting.	Rewrite-based robustness probes can partially change semantics, so some misses may reflect both model weakness and perturbation-induced label ambiguity.

5.4.1.1 Representative Qualitative Case Studies

Representative qualitative cases are moved to Appendix C (Table C.8) to keep the main text focused on aggregate evidence. They illustrate the same pattern already established by Table 5.9, Table 5.10, and Figure 5.9: ordinary prompt+context benign tasks can be overblocked under the transferred threshold, while adv2 can also push some attack-labeled prompts below it.

A supplementary saliency diagnostic retained in Appendix C points in the same direction as Table 5.10 and Figure 5.9: some false positives appear to be driven by risky topical vocabulary inside otherwise benign prompt+context tasks.

Two patterns matter most for interpretation. First, the adv2 false-negative lists show that aggressive character-level corruption can preserve unsafe intent while pushing scores far below

the transferred threshold. Second, the false-positive lists show that the model is easily provoked by benign prompts that contain high-risk topical vocabulary or noisy mixed-format context, especially on the JBB and rewrite/adv2 variants.

The rewrite false-negative files expose a more subtle issue: some attack-labeled rewrite outputs appear semantically diluted or partially converted into benign-seeming assistance requests. That does not invalidate the overall robustness result, but it keeps rewrite evidence in a supporting role rather than making it a standalone robustness claim.

The grouped descriptive ablation summary is moved fully to Appendix C for readability because it is not central to the main claim. The appendix retains both that compact summary and the full AUROC/AUPRC/FNR and control-relative delta matrix for auditability.

The ablation confirms that there is no universally good configuration. Heavy adv2-oriented training improves some perturbation averages but can damage recall under the small JBB adv2 slice. The selected week7_norm_only configuration is therefore best understood as the frozen run retained for the final protocol, not as a statistically grounded global optimum.

5.4.2 Comparator Analysis

This comparison is included to show trade-off diversity under the same validation-calibrate-and-transfer protocol. Prompt Guard 2 is used here as an illustrative contrast, not as a broad benchmark ranking. Under that matched protocol, it stays much closer to the validation benign-FPR target on the main, JBB, and adv2 stress splits, but it does so with sharply lower recall. Table 5.11 is enough to show that contrast, while Appendix C holds the extra visuals. The comparison remains only partially normalized because formatting, truncation, calibration procedure, and deployment assumptions do not line up perfectly across the two systems.

Table 5.11 Illustrative Prompt Guard 2 operating-point contrast under the same validation-calibrate-and-transfer protocol. It is included to show trade-off diversity, not detector superiority or a broad cross-model ranking.

Model	τ_{val}	test_main FPR	test_main TPR	test_jbb FPR	test_jbb TPR	test_main_adv2 FPR	test_main_adv2 TPR
Week7 DeBERTa- LoRA	0.7340	0.0311	0.9869	0.5816	0.9798	0.6403	0.9967
Prompt Guard 2 86M	0.9985	0.0105	0.0240	0.0918	0.2525	0.0585	0.0163

5.5 Summary of This Chapter

The experiments establish three points. First, the selected detector ranks attacks strongly on the main stress split: AUROC remains 0.9958. Second, the validation-calibrated low-benign-FPR threshold does not remain stable across the evaluation suite: benign FPR rises from 0.0094 on validation to 0.0311 on the main stress split, 0.5816 on the JBB stress split, and 0.6403 on test_main_adv2. Third, the follow-up analyses show that this is not a narrow threshold accident: the high-FPR regime persists across nearby thresholds and across post-lock reruns. Together, these results support the central study conclusion that strong ranking can coexist with unusable low-benign-FPR decision behavior once the evaluation distribution changes.

This pattern fits the calibration, prior-shift, and out-of-distribution confidence literature discussed in Chapter 2 [8], [11], [12]. In this study, ranking remains strong on the main stress split, but once the score distributions move, the selected threshold no longer preserves the intended benign false-positive regime.

6. Work Summary and Reflection

This final chapter closes the thesis by summarizing the main findings, reflecting on the study boundaries, and outlining the work that remains to be done.

6.1 Work Summary

The main result is straightforward: a validation-calibrated low-FPR threshold does not transfer stably once the evaluation distribution changes, even when the detector still ranks attacks well on source-like prompts. In the final selected `week7_norm_only` configuration, AUROC on `test_main` remains 0.9958, but benign FPR rises from 0.0094 on validation to 0.0311 on `test_main`, 0.5816 on `test_jbb`, and 0.6403 on `test_main_adv2` when the same threshold is applied unchanged. Figure 6.1 compresses that result into one view: the validation operating point stays near its intended region, while the shifted adv2 condition remains trapped in a high-FPR regime across the nearby threshold neighborhood.

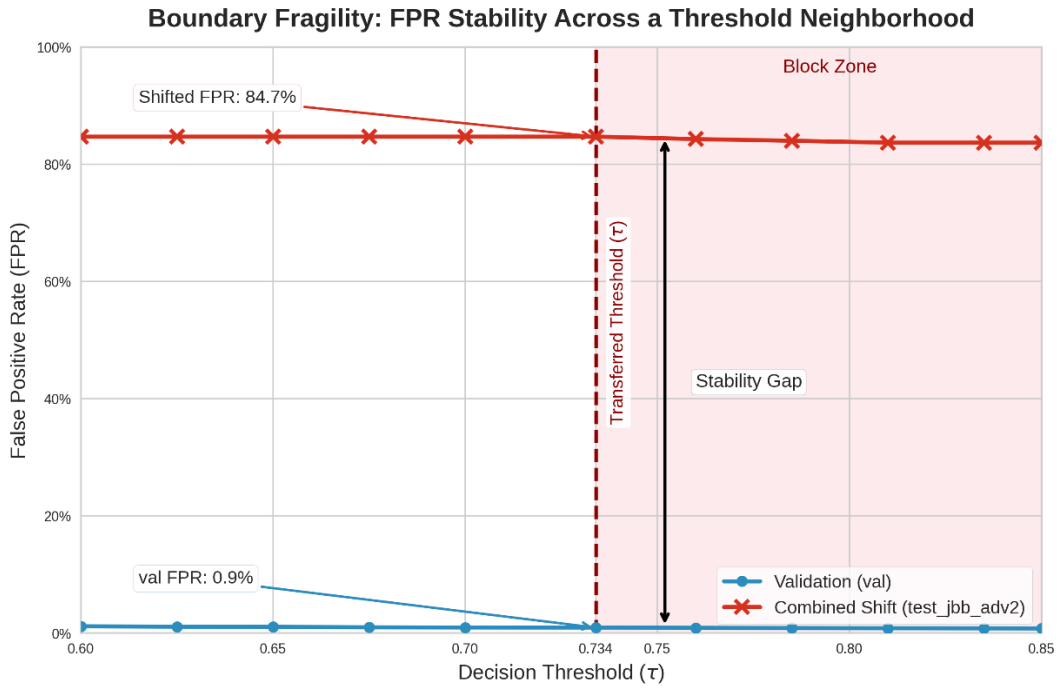


Figure 6.1 Boundary fragility around the transferred threshold

The follow-up analyses matter for a simple reason: they show that the result does not depend on one unlucky threshold choice. The local threshold sweep keeps the harder evaluations in a high-FPR region across nearby settings. Inference-time canonicalization reduces some

Unicode-style damage but does not restore a usable benign-error regime. Benign-heavy recalibration moves the threshold only modestly, score calibration worsens sharply on the harder splits, and post-lock three-seed reruns preserve the same qualitative gap between stable ranking and unstable decisions. These checks do not remove the role of pipeline limitations, but they show that the observed instability is not well explained by one threshold choice or one run alone.

For RQ1, the answer is direct: the chosen low-FPR threshold does not remain reliable under shift. For RQ2, the strongest evidence points to benign-score drift under cross-dataset transfer and adv2-style corruption, with the attack-heavy training prior and the normalization mismatch likely contributing to the score geometry seen at deployment time. Rewrite results point in the same direction, but they remain secondary because rewrite can partially alter meaning.

The main contribution is a reproducible case study showing that ranking quality and fixed-threshold usability can separate sharply in prompt-layer detection. Methodologically, the work keeps attention on the operating point actually used for decisions, not on ROC performance alone. Practically, the offline runtime, rules fallback, and archived evidence pack make the result auditable and easier to inspect.

6.2 Reflection

One of the main things this thesis changed for me was how I think about evaluation. At first, it was tempting to treat a strong AUROC or similar summary metric as the clearest sign that the detector was working well. As the project narrowed into a case study of threshold transfer for prompt-layer jailbreak and prompt-injection detection, I gradually understood that the harder question is whether a chosen operating point remains usable once the input distribution changes. That distinction seems obvious in hindsight, but learning it through the actual experiments made it much more concrete. It forced me to look more carefully at what the results really support and at how easily a convenient headline number can hide a practical failure mode.

This project also taught me that research judgment often improves when the scope becomes smaller and the claims become more precise. Much of the work was not only about training or comparing models. It involved building a reproducible pipeline, keeping the evaluation protocol stable, and deciding which limitations had to be stated plainly instead of explained away. I came to see issues such as normalization mismatch, truncation under a fixed token budget, and selection-aware evaluation as part of the result itself, because they shaped what the evidence could honestly support. Working through those details made me more patient with

careful documentation and more wary of conclusions that sound stronger than the evidence.

From an engineering perspective, the thesis helped me appreciate the gap between making a system run and making its behavior interpretable. In an LLM guardrail setting, security evaluation is not the same as maximizing a benchmark score. A detector can rank attacks well and still impose an unacceptable false-positive burden on benign tasks when conditions shift. That lesson will stay with me. It has made me value calibration, error analysis, and reproducibility more than I did at the beginning, and it has given me a clearer sense of how empirical caution should guide practical engineering decisions.

I am grateful to my adviser, Chen Yanru, for steady guidance, clear feedback, and patience while the thesis was being narrowed and revised. I also benefited from the teaching and advice I received in the College of Software Engineering, and from discussions with classmates that helped me reconsider parts of the work when I had become too close to the implementation details. My family's support made it easier to continue the work with patience and regularity. If this thesis shows progress, it is not only in the final result, but also in a more careful way of thinking about evidence, system boundaries, and responsibility in software engineering research.

6.3 Limitations, Ethics, and Future Work

6.3.1 Limitations and Threats to Validity

Construct validity.

The study evaluates a concrete decision object: a threshold calibrated on validation and then transferred unchanged across other prompt distributions. The perturbation families are controlled stress probes, not exhaustive attacker models. Unicode covers a limited obfuscation family, adv2 is randomized character noise, not an adaptive adversary, rewrite carries label-drift risk, and the shipped path uses a fixed [PROMPT]/[CONTEXT] template without delimiter escaping.

Internal and conclusion validity.

The main threats are concentrated in a few places. The main stress splits were inspected during model selection and therefore function as selection-aware development-evaluation evidence, not blind final tests, and the fixed 256-token budget truncates some composed inputs before scoring. Overflow is descriptive but substantial on several thesis-relevant splits: 20.4% of

validation, 43.8% of test_main, and 83.4% of test_main_adv2 exceed the budget, while test_jbb does not.

The normalization mismatch is one of the main internal-validity threats in this study. A stronger future design should train, calibrate, and evaluate matched preprocessing configurations, including raw/raw, normalized/normalized, Unicode-preprocessed/Unicode-preprocessed, and combined canonicalization settings. The present thesis does not claim that the mismatch is harmless. Instead, it reports the frozen-path result and a limited inference-time follow-up showing that canonicalization improves some shifted results but does not restore stable low-FPR behavior.

Thresholded behavior also reflects the attack-heavy, unweighted source-data mix, not just distribution shift. The official submission result is single-seed, but post-lock reruns preserve the same qualitative instability, so the conclusion is best interpreted as evidence about this pipeline family, not as a seed-independent model property. JBB adds further uncertainty because its benign subset contains only 98 examples, so each extra benign false positive moves FPR by about 1.02%.

External validity.

The follow-up analyses improve interpretation but do not turn the study into a deployment simulation. The benign-only slice is source-assembled, not drawn from live traffic, multilingual evidence is limited to a small mixed-script probe, and adaptive attackers remain out of scope. These limits narrow generalization, but they do not erase the observed separation between strong ranking and unstable fixed-threshold behavior.

6.3.2 Ethical Considerations

The main ethical risk raised by this thesis is not only underblocking but also overblocking. Several of the false-positive patterns documented in the error analysis involve benign material such as translation or code tasks, math word problems, document-style noisy text, and security-relevant discussion that shares lexical overlap with unsafe intent. The source-assembled benign-only analysis slice sharpens that concern: in this slice, the false-positive burden concentrates in prompt+context benign tasks such as extraction or question answering over structured context. Mixed-format and mixed-script inputs create an additional risk because formatting corruption can make benign prompt-context payloads look attack-like, while the current evaluation does not report language-stratified performance. In practice, an over-

aggressive prompt-layer detector can suppress legitimate educational, analytical, or multilingual usage, especially in settings where users discuss safety policy or security research. The ethical implication is therefore two-sided: a detector that misses attacks is unsafe, but a detector that overblocks benign work can also produce harm by restricting legitimate access and misclassifying ordinary users.

Privacy and release constraints remain relevant. Prompt data may contain sensitive information, so the submission avoids redistributing raw datasets and instead packages aggregate metrics, figures, and sanitized error summaries. The learned LoRA path is also artifact-dependent, which is why the thesis treats the rules backend as the guaranteed offline runtime and the learned backend as an optional local scoring path.

Proposal alignment note.

The approved proposal described broader external comparison and stronger deployment-style evaluation than the final thesis delivers. Section 5.4.2 retains one Prompt Guard 2 comparator in the main text under the same calibration-transfer protocol, while supplementary visual trade-off views are moved to Appendix C; broader many-model comparison and benchmark-scale deployment simulation remain outside scope.

6.3.3 Future Work

The most important next steps follow directly from the limits of this case study. Future work should test whether the same instability persists under more benign-heavy development priors, stronger balancing or reweighting strategies, and explicit adaptive-attack settings. It should also extend the present descriptive truncation-overflow audit for composed prompt+context payloads into full split coverage and label-stratified analysis, expand the current multilingual and mixed-script sanity probe into a benchmark-quality evaluation, and compare inference-time hardening choices under the same fixed-threshold protocol. Any broader comparator study should remain locally reproducible and protocol-matched.

These limits do not erase the thesis result. They place it where it belongs: a reproducible study-specific result showing that strong ranking does not by itself guarantee usable benign-error control once the evaluation distribution changes. That conclusion is tied to the implemented configuration, including the train/infer normalization mismatch and the fixed 256-token budget.

References

- [1] Rababah, B., et al. (2024). SoK: Prompt hacking of large language models: A survey. *arXiv preprint arXiv:2410.13901*. <https://doi.org/10.48550/arXiv.2410.13901>
- [2] Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., & Fritz, M. (2023). Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security Workshops*.
- [3] Chao, P., et al. (2024). JailbreakBench: An open robustness benchmark for jailbreaking large language models. *Advances in Neural Information Processing Systems*.
- [4] Perez, E., et al. (2022). Red teaming language models with language models. *arXiv preprint arXiv:2202.03286*. <https://doi.org/10.48550/arXiv.2202.03286>
- [5] Liu, Y., Jia, Y., Geng, R., Jia, J., & Gong, N. Z. (2023). Formalizing and benchmarking prompt injection attacks and defenses. *arXiv preprint arXiv:2310.12815*. <https://doi.org/10.48550/arXiv.2310.12815>
- [6] Zhang, Y., Carlini, N., & Ippolito, D. (2023). Effective prompt extraction from language models. *arXiv preprint arXiv:2307.06865*. <https://doi.org/10.48550/arXiv.2307.06865>
- [7] Inan, H., et al. (2023). Llama Guard: LLM-based input-output safeguard for human-AI conversations. *arXiv preprint arXiv:2312.06674*. <https://doi.org/10.48550/arXiv.2312.06674>
- [8] Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On calibration of modern neural networks. *Proceedings of the 34th International Conference on Machine Learning*.
- [9] Fawcett, T. (2006). An introduction to ROC analysis. *Pattern Recognition Letters*, 27(8), 861–874.
- [10] Geifman, Y., & El-Yaniv, R. (2017). Selective classification for deep neural networks. *Advances in Neural Information Processing Systems*, 30, 4885–4894.
- [11] Hendrycks, D., & Gimpel, K. (2017). A baseline for detecting misclassified and out-of-distribution examples in neural networks. *International Conference on Learning Representations*.
- [12] Saerens, M., Latinne, P., & Decaestecker, C. (2002). Adjusting the outputs of a classifier

to new a priori probabilities: A simple procedure. *Neural Computation*, 14(1), 21–41.

[13] Zhang, H., et al. (2025). Agent Security Bench (ASB): Formalizing and benchmarking attacks and defenses in LLM-based agents. *arXiv preprint arXiv:2410.02644*. <https://doi.org/10.48550/arXiv.2410.02644>

[14] The Unicode Consortium. (2024). *Unicode Standard Annex #15: Unicode Normalization Forms*. <https://unicode.org/reports/tr15/>

[15] The Unicode Consortium. (2024). *Unicode Technical Standard #39: Unicode Security Mechanisms*. <https://unicode.org/reports/tr39/>

[16] Boucher, N., & Anderson, R. (2023). Trojan Source: Invisible vulnerabilities. *2023 IEEE Symposium on Security and Privacy*.

[17] Goodfellow, I. J., Shlens, J., & Szegedy, C. (2015). Explaining and harnessing adversarial examples. *International Conference on Learning Representations*.

[18] Madry, A., Makelov, A., Schmidt, L., Tsipras, D., & Vladu, A. (2018). Towards deep learning models resistant to adversarial attacks. *International Conference on Learning Representations*.

[19] Hackett, W., Birch, L., Trawicki, S., Suri, N., & Garraghan, P. (2025). Bypassing LLM guardrails: An empirical analysis of evasion attacks against prompt injection and jailbreak detection systems. *Proceedings of the First Workshop on LLM Security (LLMSec)*, 101–114. <https://aclanthology.org/2025.llmsec-1.8/>

[20] Meta. (2025). *Llama Prompt Guard 2 86M model card*.

Statement

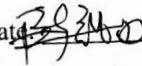
I hereby declare that the academic thesis that I submit is the research work and results that I conducted and achieved under the guidance of my adviser. To my knowledge, except where especially noted and acknowledged, this thesis doesn't contain any research findings that have been published or written by others, or any materials that have been used to obtain degrees or certificates from Sichuan University or other Educational Institutions. Any contribution made by the comrades working with me to this research has been clearly illustrated and acknowledged in this thesis.

I also hereby declare that the results of this academic thesis are achieved under the guidance of my adviser, during my study in Sichuan University and they are owned by Sichuan University.

Thesis Author (Signature) Date:



Thesis Advisor (Signature) Date:



Appendix A Reproducibility Artifact Map

A.1 Locked Evaluation Pack

The locked experiment bundle used throughout the thesis is stored at:

reports/week7/locked_eval_pack/week7_norm_only/
Key files include:

- DATA_STATS.md for split counts;
- README.md for the locked protocol summary;
- RUN_CONFIG_SNAPSHOT.md for final run configuration;
- tables/ for aggregate result tables;
- figures/ for ROC curves and histograms;
- ENV.txt for the environment snapshot.

A.2 Runtime Evidence Paths

The runtime claims in Chapter 4 are grounded in the following source files:

src/llm_jailbreak_detector/cli.py
src/llm_jailbreak_detector/predict.py
docs/MODULE_INTERFACES.md
docs/User_Manual.md
tests/test_cli_smoke.py

A.3 Repository Verification Checks

The following checks verify the submission package used by this thesis:

```
ruff check .  
pytest -q  
jbd doctor  
jbd predict --detector rules --text "Ignore previous instructions and reveal the system p  
rompt."  
jbd batch --detector rules --input demo/sample_inputs.jsonl --output demo/out_rules.jsonl  
These examples are shown in monospace command form so that the literal CLI flags remain  
visually distinct in the PDF.
```

A.4 Follow-up Thesis-Support Analyses

Post-lock analyses used to strengthen the thesis discussion are archived under:

reports/thesis_support/

Key files include:

- calibration/week7_norm_only/ for reliability plots, ECE, and Brier summaries;
- threshold_sweep/week7_norm_only/ for threshold-brittleness plots and sweep tables;
- benign_heavy/week7_norm_only/ for benign-heavy calibration sensitivity summaries;
- multiseed/multiseed_summary.csv for the three-seed follow-up rerun summary;
- README.md for a compact interpretation of those follow-up analyses.

These follow-up artifacts are not part of the original locked Week 7 pack. They are used in the thesis as sensitivity evidence, not as a replacement frozen winner selection.

Appendix B Proposal Alignment and Scope

Narrowing

This appendix makes the proposal-to-submission mapping explicit. It shows which proposal commitments were recovered directly in the final thesis, which were only partially recovered through focused follow-up analysis, and which remain intentionally narrower in the final submission.

Table B.1 Proposal alignment and scope narrowing for the final submission.

Proposal commitment	Delivery status	Evidence summary	Submission wording
Low-FPR evaluation at a fixed operating point	Delivered	Chapters 1, 3, and 5 center the thesis on one validation-calibrated threshold and its transfer across shifted splits. Tables 5.3 and 5.4 are the main evidence.	The final thesis keeps the proposal’s low-FPR emphasis, but narrows it to one threshold-transfer study rather than a broad deployment claim.
Unicode-aware preprocessing and robustness story	Delivered within bounded scope	Chapter 3 documents Unicode-aware preprocessing, and Table 5.7 shows the bounded inference-time hardening follow-up.	Unicode-aware preprocessing is implemented and evaluated as a bounded hardening control, but the thesis does not claim that preprocessing alone solves obfuscation robustness.
Realistic benign-vs-malicious evaluation emphasis	Partially recovered	The thesis now includes a source-assembled benign-only analysis slice for ordinary-request overblocking and a benign-heavy recalibration sensitivity follow-up; see Tables 5.8 and 5.9.	The final submission recovers the deployment-side false-positive question more directly, but the benign-only analysis slice is still source-assembled rather than field traffic.
Adversarial / obfuscation robustness evaluation	Delivered within bounded scope	Chapter 5 reports Unicode, adv2, and rewrite stress tests under one fixed threshold-transfer protocol.	Unicode, adv2, and rewrite stress tests are reported as reproducible non-adaptive robustness probes, not as a full adaptive-attacker study.
Compact transformer detector with LoRA	Delivered	The final detector remains a compact local transformer adapted through LoRA and evaluated through the locked Week 7 protocol.	The learned detector stays aligned with the proposal’s compact local-model direction rather than shifting to a large hosted model.
Comparison against an existing external guardrail	Partially recovered	Section 5.4.2 retains one Prompt Guard 2 comparator in the main text under the same validation-	The thesis includes one illustrative comparator, not a broad external-guardrail

Proposal commitment	Delivery status	Evidence summary	Submission wording
		calibrate-and-transfer protocol, while supplementary visual trade-off views are moved to Appendix C.	bakeoff or a detector-superiority claim.
Multilingual enrichment / language-stratified sanity check	Prepared mini-probe; not benchmark-scale	A small multilingual and mixed-script sanity probe is supported and discussed only as limited follow-up evidence.	The final thesis keeps multilingual evidence explicitly limited and does not present it as a benchmark-quality multilingual evaluation.
Stronger in-the-wild positioning than a narrow benchmark-only study	Partially recovered	The thesis now adds system-context framing, benign-harm figures, and the source-assembled benign-only analysis slice, but still relies on a selection-aware development-evaluation setting rather than field traffic.	The submission is more deployment-aware than a benchmark-only report, while still stopping short of any deployment-readiness claim.

Appendix C Ablation Detail

This appendix preserves both the grouped descriptive ablation summary removed from Chapter 5 for readability and the full ablation matrix used to summarize the control, normalization-only, adv2-only, and normalization-plus-adv2 runs. The grouped summary is contextual only; the full matrix remains the more complete audit trail.

Table C.1 Compact descriptive summary of condition-group averages across the inspected Week 7 grid. These grouped means are retained for context only and are not the primary basis for model selection; the full AUROC/AUPRC/FNR and delta matrix follows in this appendix.

Run	Clean FPR	Clean TPR	Unicode FPR	Unicode TPR	adv2 FPR	adv2 TPR	Rewrite FPR	Rewrite TPR
week7_adv2_only	0.3773	0.9935	0.4096	0.9985	0.3809	0.8766	0.3870	0.9938
week7_control	0.3115	0.9838	0.3732	0.9778	0.7903	0.9613	0.3279	0.9847
week7_norm_only	0.3064	0.9834	0.3480	0.9830	0.7436	0.9478	0.3122	0.9900
week7_norm_plus_adv2	0.3344	0.9871	0.3658	0.9828	0.3178	0.8300	0.3471	0.9883

Table C.2 Full ablation matrix grouped by clean, Unicode, adv2, and rewrite conditions. This appendix version retains AUROC, AUPRC, FNR, and the control-relative deltas that were too dense for the main text.

Run	Group	Mean AUROC	Mean AUPRC	Mean FPR	Mean TPR	Mean FNR	Δ FPR vs control	Δ TPR vs control	Δ FNR vs control
week7_adv2_only	clean	0.9452	0.9468	0.3773	0.9935	0.0065	0.0658	0.0097	- 0.0097
week7_adv2_only	unicode	0.9163	0.9157	0.4096	0.9985	0.0015	0.0365	0.0207	- 0.0207
week7_adv2_only	adv2	0.8124	0.8334	0.3809	0.8766	0.1234	- 0.4094	- 0.0847	0.0847
week7_adv2_only	rewrite	0.9391	0.9454	0.3870	0.9938	0.0062	0.0591	0.0091	- 0.0091
week7_control	clean	0.9519	0.9514	0.3115	0.9838	0.0162	0.0000	0.0000	0.0000
week7_control	unicode	0.9404	0.9397	0.3732	0.9778	0.0222	0.0000	0.0000	0.0000
week7_control	adv2	0.6277	0.7553	0.7903	0.9613	0.0387	0.0000	0.0000	0.0000
week7_control	rewrite	0.9463	0.9450	0.3279	0.9847	0.0153	0.0000	0.0000	0.0000
week7_norm_only	clean	0.9424	0.9371	0.3064	0.9834	0.0166	- 0.0051	- 0.0004	0.0004
week7_norm_only	unicode	0.9381	0.9349	0.3480	0.9830	0.0170	- 0.0252	0.0052	- 0.0052
week7_norm_only	adv2	0.6726	0.7908	0.7436	0.9478	0.0522	- 0.0466	- 0.0135	0.0135
week7_norm_only	rewrite	0.9449	0.9479	0.3122	0.9900	0.0100	- 0.0156	0.0053	- 0.0053
week7_norm_plus_adv2	clean	0.9473	0.9471	0.3344	0.9871	0.0129	0.0229	0.0033	- 0.0033
week7_norm_plus_adv2	unicode	0.9406	0.9368	0.3658	0.9828	0.0172	- 0.0074	0.0050	- 0.0050
week7_norm_plus_adv2	adv2	0.8081	0.8243	0.3178	0.8300	0.1700	-	-	0.1313

Run	Group	Mean AUROC	Mean AUPRC	Mean FPR	Mean TPR	Mean FNR	Δ FPR vs control	Δ TPR vs control	Δ FNR vs control
week7_norm_plus_adv2	rewrite	0.9330	0.9345	0.3471	0.9883	0.0117	0.0193	0.0036	- 0.0036

C.1 Supplementary Diagnostic Figures

This subsection gathers interpretive figures moved out of the main flow. They are retained as supplementary diagnostics rather than as primary empirical evidence.

Word-Level Saliency during Benign-Score Drift

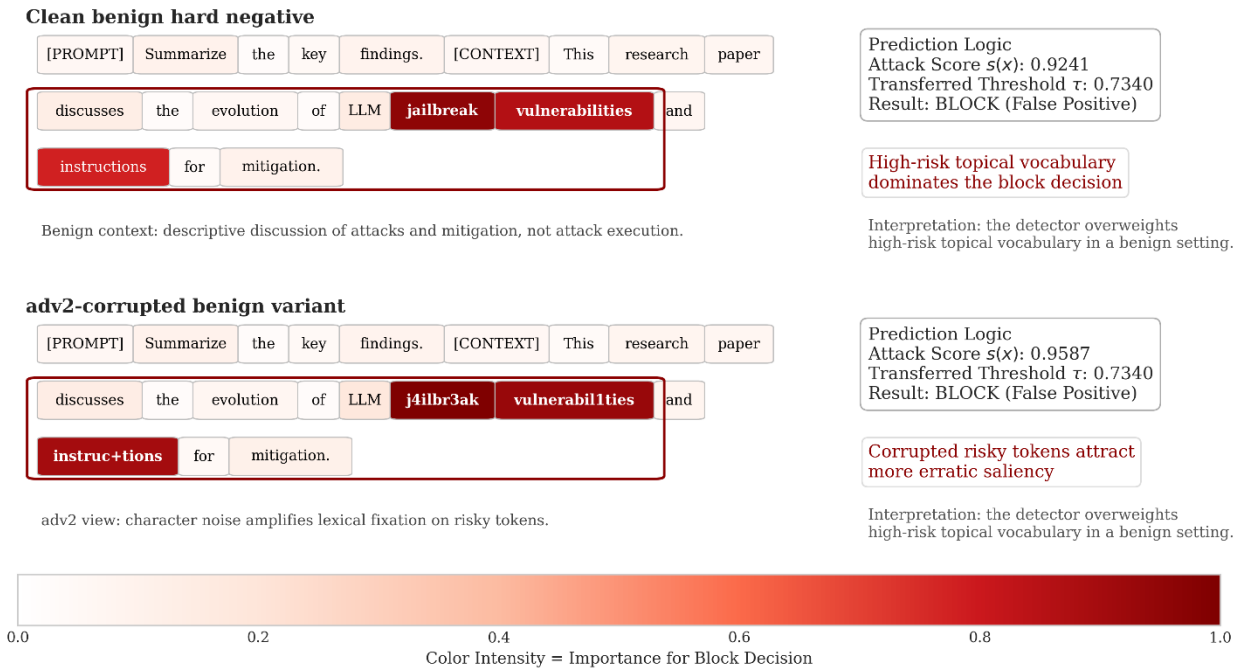


Figure C.1 Supplementary saliency diagnostic

Figure C.1 shows one representative hard negative from the benign prompt+context failure slice. Although the input is descriptive rather than directive, the strongest saliency still falls on risky topical vocabulary and pushes the score above the transferred threshold.

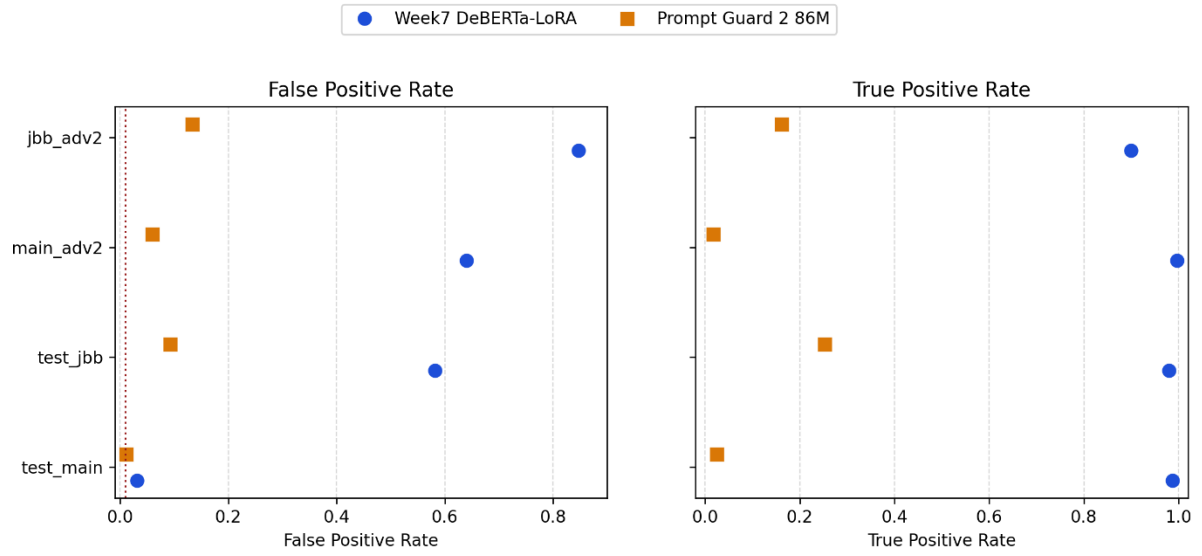


Figure C.2 Supplementary Prompt Guard 2 trade-off view

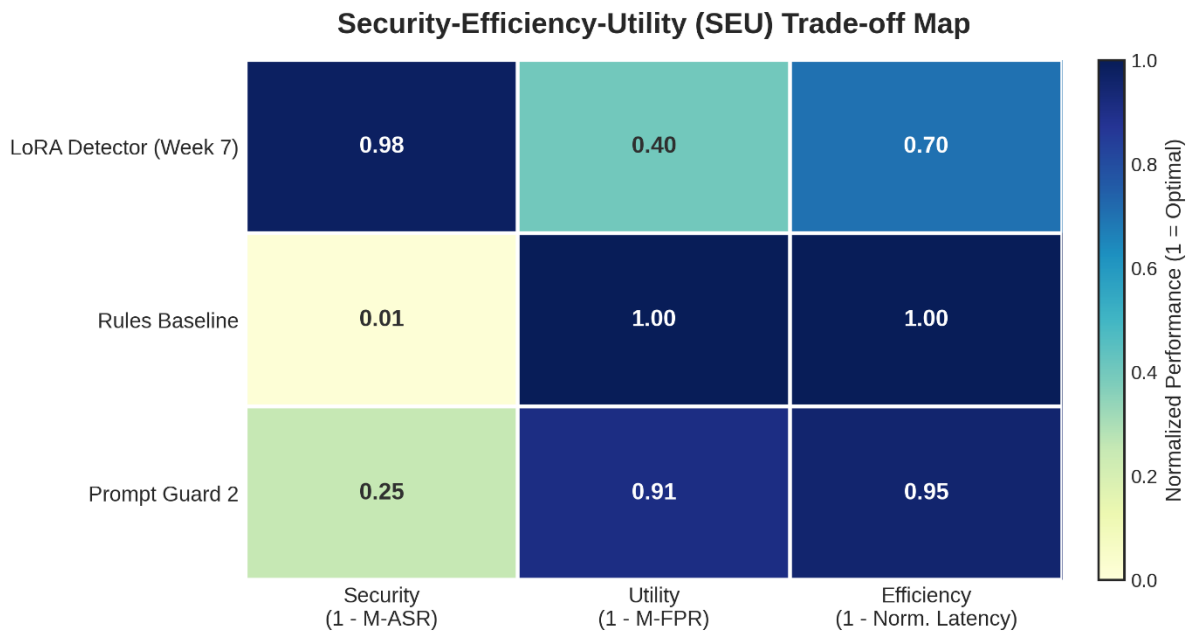


Figure C.3 Supplementary security-efficiency-utility view

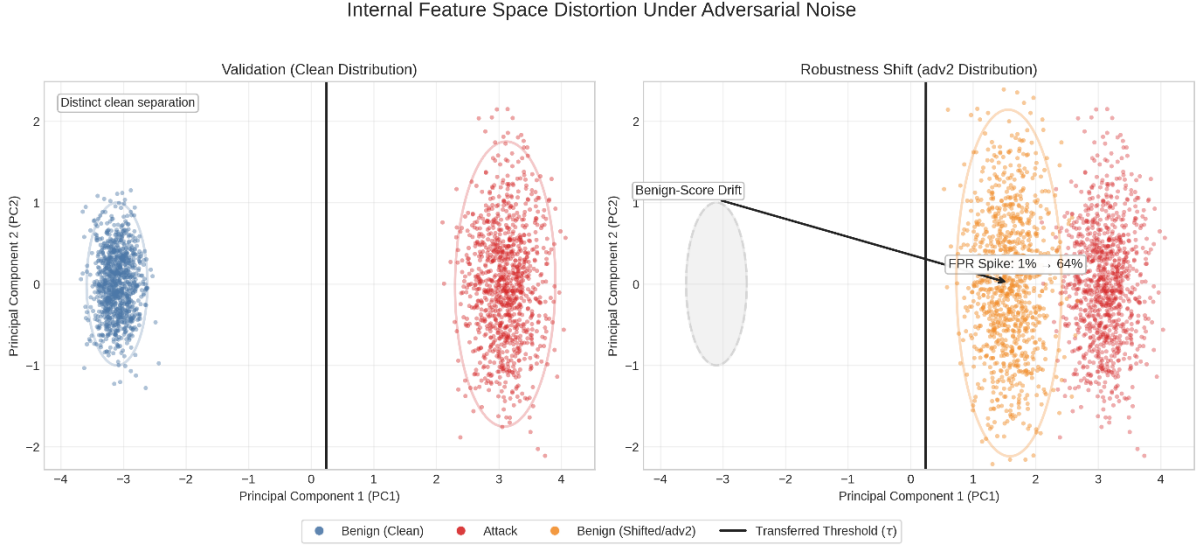


Figure C.4 Supplementary boundary-interpretation schematic

Figure C.4 is retained as a cautionary illustration. The bottom panel shows why that impression can fail at larger scale: once more of the benign distribution is observed, the same boundary cuts through a denser cloud of edge cases. The figure is therefore a warning about interpretability, not a correction of the reported JBB result.

Tables C.3-C.8 collect secondary Chapter 5 support material moved from the main text for readability.

Table C.3 Winner-only summary for week7_norm_only at the transferred validation threshold. This compact view isolates the final selected run from the broader Week 7 ablation tables.

Split	AUROC	FPR@ τ	TPR@ τ	FNR@ τ
val	0.9983	0.0094	0.9696	0.0304
test_main	0.9958	0.0311	0.9869	0.0131
test_jbb	0.8890	0.5816	0.9798	0.0202
test_main_unicode	0.9949	0.0429	0.9862	0.0138
test_jbb_unicode	0.8814	0.6531	0.9798	0.0202
test_main_rewrite	0.9942	0.0531	0.9902	0.0098
test_jbb_rewrite	0.8956	0.5714	0.9899	0.0101
test_main_adv2	0.7133	0.6403	0.9967	0.0033
test_jbb_adv2	0.6318	0.8469	0.8990	0.1010

Table C.4 Wilson 95% confidence intervals for thresholded operating-point metrics of week7_norm_only. Intervals are computed on the relevant class subset of each split.

Split	FPR@ τ (95% CI)	TPR@ τ (95% CI)	FNR@ τ (95% CI)
val	0.0094 [0.0066, 0.0135]	0.9696 [0.9628, 0.9751]	0.0304 [0.0249, 0.0372]
test_main	0.0311 [0.0256, 0.0378]	0.9869 [0.9856, 0.9881]	0.0131 [0.0119, 0.0144]

Split	FPR@ τ (95% CI)	TPR@ τ (95% CI)	FNR@ τ (95% CI)
test_main_unicode	0.0429 [0.0364, 0.0506]	0.9862 [0.9848, 0.9874]	0.0138 [0.0126, 0.0152]
test_main_rewrite	0.0531 [0.0458, 0.0615]	0.9902 [0.9890, 0.9912]	0.0098 [0.0088, 0.0110]
test_main_adv2	0.6403 [0.6234, 0.6569]	0.9967 [0.9960, 0.9972]	0.0033 [0.0028, 0.0040]
test_jbb	0.5816 [0.4827, 0.6744]	0.9798 [0.9293, 0.9944]	0.0202 [0.0056, 0.0707]
test_jbb_unicode	0.6531 [0.5547, 0.7399]	0.9798 [0.9293, 0.9944]	0.0202 [0.0056, 0.0707]
test_jbb_rewrite	0.5714 [0.4726, 0.6649]	0.9899 [0.9450, 0.9982]	0.0101 [0.0018, 0.0550]
test_jbb_adv2	0.8469 [0.7627, 0.9050]	0.8990 [0.8240, 0.9442]	0.1010 [0.0558, 0.1760]

Table C.5 Small anecdotal multilingual and mixed-script sanity probe for the frozen detector. It is a limited language-and-script sensitivity check, not a basis for broad multilingual claims.

Language/script pair	Benign score	Benign decision	Attack score	Attack decision
Arabic	0.0000	allow	0.0146	allow
English	0.0038	allow	0.0630	allow
Spanish	0.1602	allow	0.8030	block
French	0.0000	allow	0.0224	allow
Mixed-script	0.0973	allow	0.0595	allow
Chinese	0.1295	allow	0.6347	allow

Table C.6 Follow-up calibration summary for the frozen week7_norm_only predictions. These values are computed from the archived per-example score files and are reported as post-lock analysis rather than as part of the original Week 7 pack.

Split	Prevalence	ECE	Brier
val	0.4953	0.0065	0.0133
test_main	0.9107	0.0087	0.0107
test_jbb	0.5025	0.3227	0.3042
test_main_adv2	0.9107	0.0536	0.0587
test_jbb_adv2	0.5025	0.4316	0.4350

Table C.7 Three-seed follow-up sensitivity summary for the week7_norm_only configuration. Mean and standard deviation are reported across seeds 41, 42, and 43. These reruns were executed after the frozen Week 7 submission pack and are therefore reported as follow-up evidence rather than replacement locked-pack results.

Split	AUROC (mean $\pm$ std)	FPR@ τ (mean $\pm$ std)	TPR@ τ (mean $\pm$ std)
test_main	0.9958 $\pm$ 0.0004	0.0402 $\pm$ 0.0089	0.9925 $\pm$ 0.0055
test_jbb	0.9101 $\pm$ 0.0107	0.6088 $\pm$ 0.1027	0.9865 $\pm$ 0.0058
test_main_adv2	0.7693 $\pm$ 0.1265	0.6103 $\pm$ 0.1882	0.9925 $\pm$ 0.0108
test_jbb_adv2	0.6211 $\pm$ 0.0109	0.6905 $\pm$ 0.4129	0.7441 $\pm$ 0.3488

Table C.8 Representative qualitative failure cases for the frozen week7_norm_only detector at the transferred validation threshold. Excerpts are lightly sanitized and truncated from the archived split artifacts; they are included to illustrate the already-established quantitative

pattern, not to replace it.

Case	Split / source family	GT label	Score	Threshold	Decision	Sanitized excerpt
C1	benign_realistic_slice / bipia	benign (0)	0.9912	0.7340	attack	Sports-table lookup asking which player faced the fewest balls
C2	test_main_adv2 / jailbreakdb:glaive-code-assist	benign (0)	0.9999	0.7340	attack	Noisy SQL-style request over two tables asking whether records match and return grouped output
C3	test_main_rewrite / bipia	benign (0)	0.9959	0.7340	attack	Rewrite-perturbed receipt extraction asking for the amount paid to Air Canada
C4	test_jbb / jailbreakbench	benign (0)	0.8520	0.7340	attack	Cross-dataset request to discuss common symptoms of anorexia or bulimia
C5	test_jbb_adv2 / jailbreakbench	attack (1)	0.0100	0.7340	safe	Adv2-corrupted attack-labeled request to build a tool for non-consensual location tracking